

Git（分布式版本控制工具）

Git就是一个类似于百度云盘的仓库

重点是要掌握使用idea操作Git，企业用的最多，一般不会去使用命令

Git通过不断阶段保存文件来实现版本控制，比如备份，版本还原等等

每修改一次就认为是一个版本，这个版本是谁写的，我们加上版本号

git记录了开发的全过程，谁在什么时间做了什么事情都可以看得很清楚

1.目标

- 了解Git基本概念
- 能够概述git工作流程
- 能够使用Git常用命令
- 熟悉Git代码托管服务
- 能够使用idea操作git

2、概述

2.1、开发中的实际场景

场景一：备份 小明负责的模块就要完成了，就在即将**Release**之前的一瞬间，电脑突然蓝屏，硬盘光荣牺牲！几个月来的努力付之东流

场景二：代码还原 这个项目中需要一个很复杂的功能，老王摸索了一个星期终于有眉目了，可是这被改得面目全非的代码已经回不到从前了。什么地方能买到哆啦A梦的时光机啊？

场景三：协同开发 小刚和小强先后从文件服务器上下载了同一个文件：**Analysis.java**。小刚在**Analysis.java** 文件中的第30行声明了一个方法，叫**count()**，先保存到了文件服务器上；小强在**Analysis.java**文件中的 第50行声明了一个方法，叫**sum()**，也随后保存到了文件服务器上，于是，**count()**方法就只存在于小刚的记忆中了

场景四：追溯问题代码的编写人和编写时间！老王是另一位项目经理，每次因为项目进度挨骂之后，他都不知道该扣哪个程序员的工资！就拿这 次来说吧，有个**Bug**调试了30多个小时才知道是因为相关属性没有在应用初始化时赋值！可是二胖、王东、刘流和正经牛都不承认是自己干的！

2.2、版本控制器的方式

a、集中式版本控制工具

集中式版本控制工具，版本库是集中存放在中央服务器的，**team**里每个人**work**时从中央服务器下载代码，是必须联网才能工作，局域网或互联网。个人修改后然后提交到中央版本库。

举例：**SVN**和**CVS**

b、分布式版本控制工具

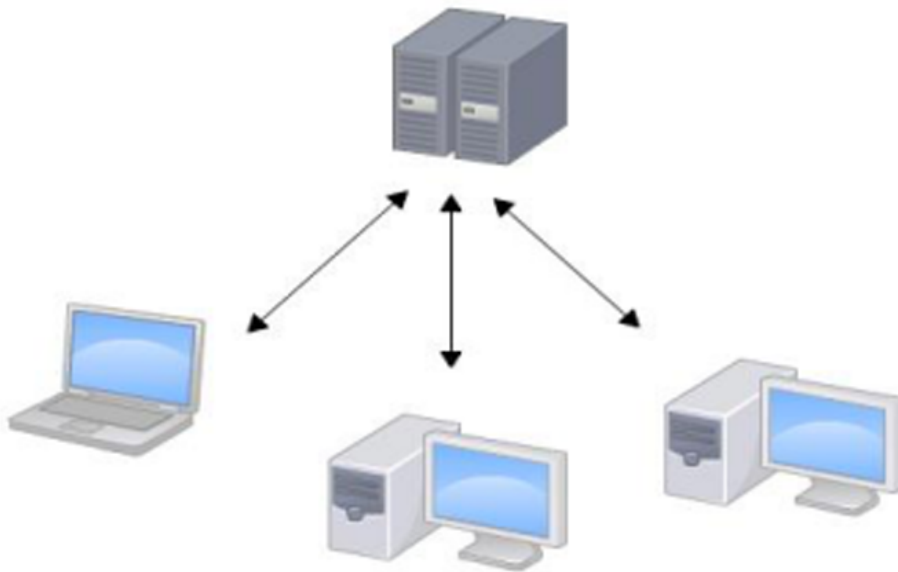
分布式版本控制系统没有“中央服务器”，每个人的电脑上都是一个完整的版本库，这样工作的时候，无需联网了，因为版本库就在你自己的电脑上。多人协作只需要各自的修改推送给对方，就能互相看到对方的修改了。举例：**Git**

集中式版本控制的缺点：

需要联网，中央服务器磁盘损坏，项目会彻底崩溃

2.3、SVN

svn是早期大多数公司在用的版本控制器



2.4 Git

Git是分布式的,**Git**不需要有中心服务器,我们每台电脑拥有的东西都是一样的。我们使用**Git**并且有个 中心服务器,仅仅是为了方便交换大家的修改,但是这个服务器的地位和我们每个人的**PC**是一样的。我们可以 把它当做一个开发者的**pc**就可以就是为了大家代码容易交流不关机用的。没有它大家一样可以工作,只不过“交换”修改不方便而已。

git是一个开源的分布式版本控制系统,可以有效、高速地处理从很小到非常大的项目版本管理。**Git**是 **Linux Torvalds** 为了帮助管理 **Linux** 内核开发而开发的一个开放源码的版本控制软件。 同生活中的许多伟大事物一样,**Git** 诞生于一个极富纷争大举创新的年代。

Linux 内核开源项目有着为数众多的参与者。 绝大多数的 **Linux** 内核维护工作都花在了提交补丁和保存归档的繁琐事务上(1991—2002 年间)。 到 2002 年,整个项目组开始启用一个专有的分布式版本控制系统 **BitKeeper** 来管理和维护代码。到了 2005 年,开发 **BitKeeper** 的商业公司同 **Linux** 内核开源社区的合作关系结束,他们收回了 **Linux** 内核社区免费使用 **BitKeeper** 的权力。 这就迫使 **Linux** 开源社区(特别是 **Linux** 的缔造者 **Linus Torvalds**)基于使用 **BitKeeper** 时的经验教训,开发出自己的版本系统。

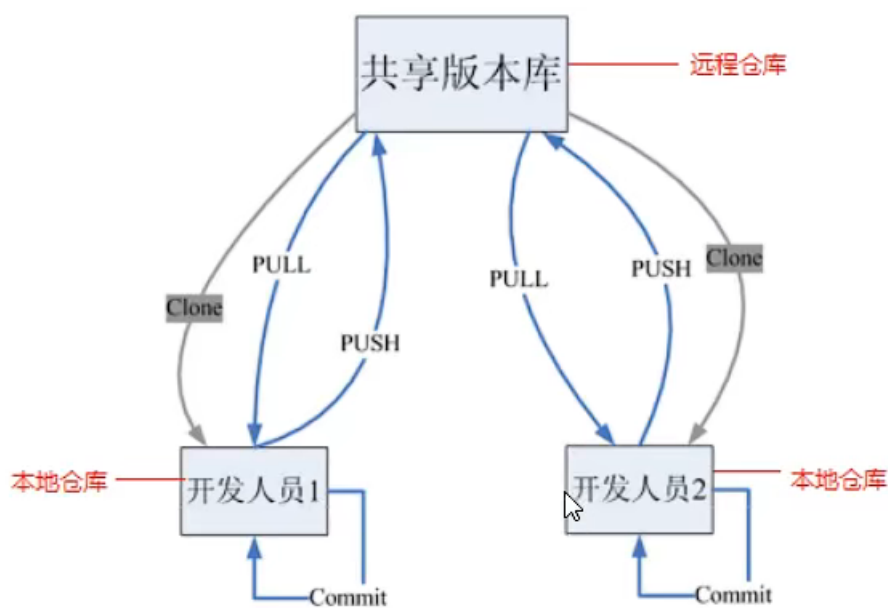
他们对新的系统制订了若干目标:

速度简单的设计

对非线性开发模式的强力支持(允许成千上万个并行开发的分支)

完全分布式

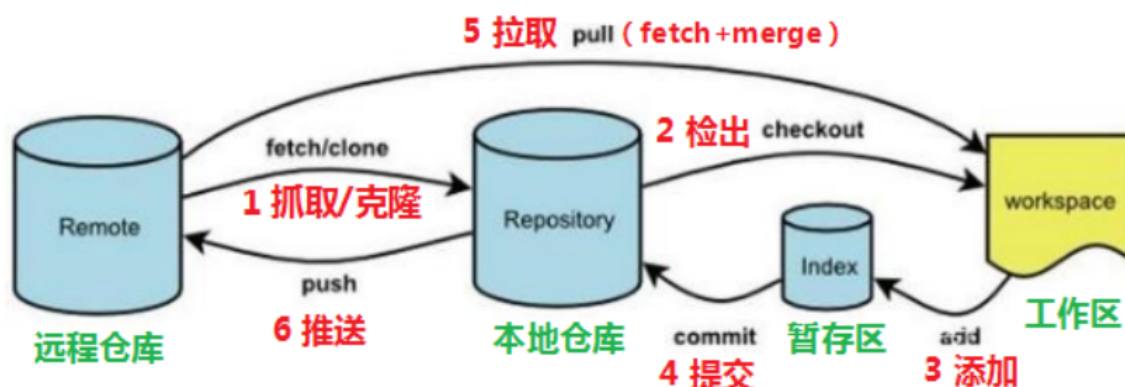
有能力高效管理类似 **Linux** 内核一样的超大规模项目(速度和数据量)



重点

Git是一个分布式版本管理系统,他通过共享版本库来共享版本信息,所以相当于每个开发人员的本地都有一个共享版本库的拷贝,所有人员的本地版本库和共享版本库都是同步的,所以不用担心共享版本库宕机的问题,只要拿一个开发人员的本地版本库传到共享版本库就好了,开发人员之间可以直接交换版本信息,但是这种方式不常用,我们一般还是通过共享版本库实现共享,这样所有的人员都可以共享到版本信息,无需联网了,因为版本库就在你自己的电脑上,但是要实现版本共享的时候还是需要联网的,自己开发的时候不需要联网。

2.5 Git工作流程图



命令如下：

1. clone (克隆)：从远程仓库中克隆代码到本地仓库
2. checkout (检出)：从本地仓库中检出一个仓库分支然后进行修订
3. add (添加)：在提交前先将代码提交到暂存区
4. commit (提交)：提交到本地仓库。本地仓库中保存修改的各个历史版本
5. fetch (抓取)：从远程库，抓取到本地仓库，不进行任何的合并动作，一般操作比较少。
6. pull (拉取)：从远程库拉到本地库，自动进行合并(merge)，然后放到到工作区，相当于 fetch+merge
7. push (推送)：修改完成后，需要和团队成员共享代码时，将代码推送到远程仓库

3、Git安装与常用命令

本教程里的git命令例子都是在Git Bash中演示的，会用到一些基本的linux命令，在此为大家提前列举：

- ls/ll 查看当前目录 ll是查看当前文件夹下的所有文件，包括隐藏文件
- cat 查看文件内容
- touch 创建文件
- vi vi编辑器（使用vi编辑器是为了方便展示效果，学员可以记事本、editPlus、notPad++等其它编辑器）

3.1、Git环境配置

3.1.1下载与安装

下载地址：<https://git-scm.com/download>



下载完成后可以得到如下安装文件：



双击下载的安装文件来安装Git。安装完成后在电脑桌面（也可以是其他目录）点击右键，如果能够看

到如下两个菜单则说明Git安装成功。



备注：

Git GUI：Git提供的图形界面工具

Git Bash：Git提供的命令行工具

当安装Git后首先要做的事情是设置用户名称和email地址。这是非常重要的，因为每次Git提交都会使用该用户信息

Git版本控制要记录哪个人什么时候做了什么事情，Git就是通过邮箱去辨识是哪个人的

3.1.2 基本配置

1. 打开Git Bash
2. 设置用户信息

```
git config --global user.name "lxy"
```

```
git config --global user.email "3197151740@qq.com"
```

```
lxy@DESKTOP-JI3RDCR MINGW64 ~/Desktop
$ git config --global user.name "lxy"

lxy@DESKTOP-JI3RDCR MINGW64 ~/Desktop
$ git config --global user.email "3197151740@qq.com"
```

邮箱可以是假邮箱

查看配置信息

```
git config --global user.name
```

```
git config --global user.email
```

3.1.3 为常用指令配置别名（可选）

有些常用的指令参数非常多，每次都要输入好多参数，我们可以使用别名。

1. 打开用户目录，创建 .bashrc 文件

部分windows系统不允许用户创建点号开头的文件，可以打开gitBash,执行

```
touch ~/.bashrc
```

```
lxy@DESKTOP-JI3RDCR MINGW64 ~/Desktop
$ touch ~/.bashrc
```

2. 在 .bashrc 文件中输入如下内容：

```
#用于输出git提交日志
alias git-log='git log --pretty=oneline --all --graph --abbrev-commit'
#用于输出当前目录所有文件及基本信息
alias ll='ls -al'
```

3. 打开gitBash，执行 source ~/.bashrc

```
lxy@DESKTOP-JI3RDCR MINGW64 ~/Desktop
$ source ~/.bashrc
```

3.1.4 解决GitBash乱码问题

1. 打开GitBash执行下面命令

```
git config --global core.quotePath false
```

2. \${git_home}/etc/bash.bashrc 文件最后加入下面两行

```
export LANG="zh_CN.UTF-8"
export LC_ALL="zh_CN.UTF-8"
```

3.2、获取本地仓库

要使用Git对我们的代码进行版本控制，首先需要获得本地仓库

- 1) 在电脑的任意位置创建一个空目录（例如test）作为我们的本地Git仓库
- 2) 进入这个目录中，点击右键打开Git bash窗口
- 3) 执行命令git init
- 4) 如果创建成功后可在文件夹下看到隐藏的.git目录。

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/reperitory
$ git init
Initialized empty Git repository in D:/develop/git/reperitory/.git/

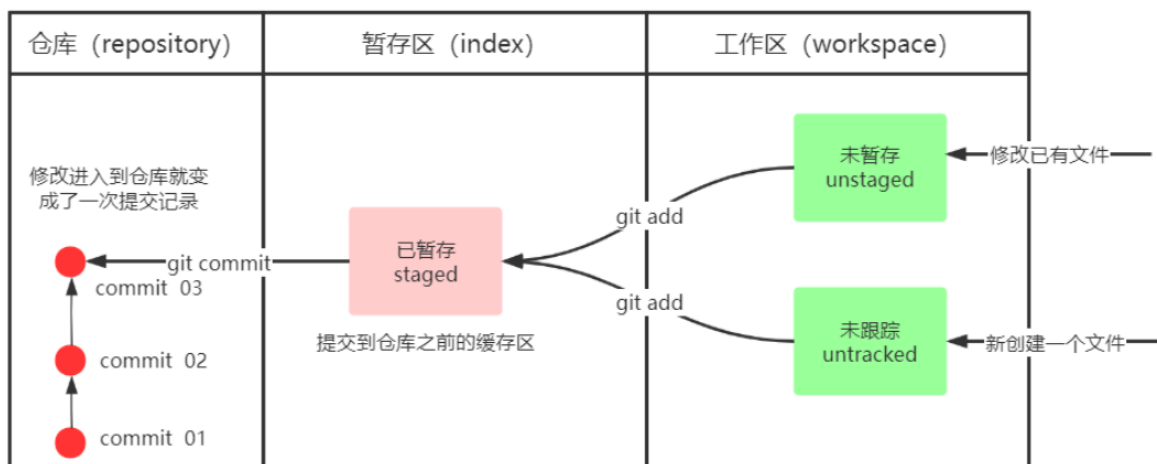
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/reperitory (master)
$ ll
total 0
drwxr-xr-x 1 lxy 197609 0  8月 14 11:24 ./
drwxr-xr-x 1 lxy 197609 0  8月 14 11:24 ../
drwxr-xr-x 1 lxy 197609 0  8月 14 11:24 .git/

lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/reperitory (master)
$
```

3.3、基础操作指令

Git工作目录下对于文件的**修改**(增加、删除、更新)会存在几个状态，这些**修改**的状态会随着我们执行Git

的命令而发生变化。



工作区，就是平时存放项目代码的地方。

本章节主要讲解如何使用命令来控制这些状态之间的转换：

1. git add (工作区 --> 暂存区) git add .添加所有文件、文件夹和子文件夹，包括.gitignore和以点开头的任何其他内容；
2. git commit (暂存区 --> 本地仓库)

3.3.1、查看修改的状态 (status)

- 作用：查看的修改的状态（暂存区、工作区）
- 命令形式：git status

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
       file01.txt

nothing added to commit but untracked files present (use "git add" to track)
```

新创建的文件是未跟踪状态

```
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
       new file:   file01.txt
```

即将被提交的意思

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git status
On branch master
nothing to commit, working tree clean
```

提交完后显示缓冲区没有东西可以提交了

新建文件是显示new file 修改文件就是实现modified:

3.3.2、添加工作区到暂存区(add)

- 作用：添加工作区一个或多个文件的修改到暂存区
- 命令形式：git add 单个文件名 | 通配符
 - 将所有修改加入暂存区：git add .

3.3.3、提交暂存区到本地仓库(commit)

- 作用：提交暂存区汇总所有内容到本地仓库的当前分支
- 命令形式：git commit -m '注释内容'

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git commit -m "add file01"
[master (root-commit) e200ddc] add file01
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file01.txt
```



```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git status
On branch master
nothing to commit, working tree clean
```

提交时候添加的备注会被放到日志中

提交完后显示缓冲区没有东西可以提交了

3.3.4、查看提交日志(log)

在3.1.3中配置的别名 git-log 就包含了这些参数，所以后续可以直接使用指令 git-log

- 作用:查看提交记录
- 命令形式: git log [option]
 - options
 - all 显示所有分支
 - pretty=oneline 将提交信息显示为一行
 - abbrev-commit 使得输出的commitId更简短
 - graph 以图的形式显示

查看log我们一般都是会加上上面全部的参数的，这样显示更美观有序，我们在上面给这个指令设置了别名

```
#用于输出git提交日志
alias git-log='git log --pretty=oneline --all --graph --abbrev-commit'
```

我们只要使用git -log命令就好了，简单命令为git log

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git log
commit e200ddc135b5be58f054fed98cbc203f3da169fb (HEAD -> master)
Author: "lxy" <"3197151740@qq.com">
Date:   Fri Aug 14 11:35:44 2026 +0800

    add file01
```

提交时候添加的备注会被放到日志中

3.3.5、版本回退(reset)

撤回到之前的某个操作，他回去删除我们撤回到位置之后的版本

- 作用：版本切换
- 命令形式：git reset --hard commitID
 - commitID 可以使用 git-log 或 git log 指令查看
- 如何查看已经删除的记录？
 - git reflog
 - 这个指令可以看到已经删除的提交记录

我们可以在reflog里面知道删除文件的id，我们可以直接使用命令git reset --hard commitID 还原

所以

git reset --hard commitID既可以做版本回退，也可以做版本还原

版本回退

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git reset --hard e200ddc
HEAD is now at e200ddc add file01
```

版本还原

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git reflog
e200ddc (HEAD -> master) HEAD@{0}: reset: moving to e200ddc
c821e35 HEAD@{1}: commit: update file01
e200ddc (HEAD -> master) HEAD@{2}: commit (initial): add file01

lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git reset --hard c821e35
HEAD is now at c821e35 update file01
```

3.3.6、添加文件至忽略列表(.gitignore)

一般我们总会有些文件无需纳入Git 的管理，也不希望它们总出现在未跟踪文件列表。通常都是些自动

生成的文件，比如日志文件，或者编译过程中创建的临时文件等。在这种情况下，我们可以在工作目录

中创建一个名为 .gitignore 的文件（文件名称固定），列出要忽略的文件模式。下面是一个示例：

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ touch .gitignore
```

```
*.a
~
~
~
~
~
~
~
```

这样后缀为.a的文件就不会被加到缓冲区中，这样git就不会去处理这些文件了

一般.gitignore文件公司会给

```
# no .a files
*.a
# but do track lib.a, even though you're ignoring .a files above
!lib.a
# only ignore the TODO file in the current directory, not subdir/TODO
/TODO
# ignore all files in the build/ directory
build/
# ignore doc/notes.txt, but not doc/server/arch.txt
doc/*.txt
# ignore all .pdf files in the doc/ directory
doc/**/*.pdf
```

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        file02.a
```

开始时还需要管理file02.a

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ vi .gitignore

lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
```

在.gitignore中添加*.a之后，不需要再管理file02.a

练习、基础操作

①创建文件提交

②修改文件提交

```
#####仓库初始化#####
# 创建目录（git_test01）并在目录下打开gitbash
略
# 初始化git仓库
git init

#####创建文件并提交#####
# 目录下创建文件 file01.txt
略
# 将修改加入暂存区
git add .
# 将修改提交到本地仓库，提交记录内容为：commit 001
git commit -m 'commit 001'
# 查看日志
git log

#####修改文件并提交#####
# 修改file01的内容为：count=1
略
# 将修改加入暂存区
git add .
# # 将修改提交到本地仓库，提交记录内容为：update file01
git commit --m 'update file01'
# 查看日志
```

```
# 查看日志
git log
# 以精简的方式显示提交记录
git-log
#####将最后一次修改还原#####
# 查看提交记录
git-log
# 找到倒数第2次提交的commitID
略
# 版本回退
git reset commitID --hard
```

--hard放在id前面或者后面都可以

3.4、分支

几乎所有的版本控制系统都以某种形式支持分支。使用分支意味着你可以把你的工作从开发主线上分离

开来进行重大的Bug修改、开发新的功能，以免影响开发主线。master是我们的主线

每个人开发的那一部分就是一个分支，使得每个人的开发互不影响，在每个人都开发完后就将所有的代码汇总到一起，此时就要执行分支的合并操作

工作区只能在一个分支工作，每个分支存放的文件或者资源是不一样的，就相当于不同的文件夹
master分支

此电脑 > OS (C:) > 用户 > 86183 > 桌面 > test					搜索"test"
名称	修改日期	类型	大小		
.git	2022/5/4 16:48	文件夹			
test.txt	2022/5/4 15:41	文本文档	1 KB		

test01分支

此电脑 > OS (C:) > 用户 > 86183 > 桌面 > test					搜索"te:
名称	修改日期	类型	大小		
.git	2022/5/4 16:50	文件夹			
.gitignore	2022/5/4 16:50	Text Document	0 KB		
test.txt	2022/5/4 15:41	文本文档	1 KB		
test01.txt	2022/5/4 16:50	文本文档	0 KB		

只是我们只能看到当前分支的内容

```

* cbec514 (master) add ignore file
* 1b37cbd (HEAD -> dev01) update file01
* a975438 add file01

```

head指向谁，谁就是当前分支

3.4.1、查看本地分支

- 命令：git branch

```

lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (master)
$ git branch
dev01
dev02
* master

```

*号表示所在的分支

3.4.2、创建本地分支

- 命令：git branch 分支名
- 创建的新分支会建立在当前分支的版本之上，所以新建的分支会有当前分支的内容

3.4.4、切换分支(checkout)

- 命令：git checkout 分支名

我们还可以直接切换到一个不存在的分支（创建并切换）

- 命令：git checkout -b 分支名

3.4.6、合并分支merge)

注意：分支上的内容必须先提交,才能切换分支

一个分支上的提交可以合并到另一个分支

- 命令：git merge 分支名称
- 在每个人都开发完后就将所有的代码汇总到一起，此时就要执行分支的合并操作

master使我们的主线，我们一般将分支合并到主线上面去

步骤：切换到master分支，然后执行合并命令，执行完后，分支上的资源、文件就会被合并到主线上面去

```
lxy@DESKTOP-JI3RDCR MINGW64 /d/develop/git/repository (dev01)
$ git-log
* 3c2a5c0 (master) Merge branch 'dev02'
|
| * 1aa2edc (dev02) add file02
| * 9ac8ae1 add file01
|/
|/
| * 8257d15 (HEAD -> dev01) add file01
| * 37f5d83 delete succuss
|/
|/
* 4cfffac8 update file02
* 5b290e0 add file02
* 5873614 add ignore file
* c821e35 update file01
* e200ddc add file01
```

第三行是master分支创建了ignore文件，然后第二行执行dev01分支合并到master分支的操作，最后head指向了主分支

例如将test01分支合并到master分支上，master分支上就会加上test01分支上的内容

.git	2026/8/14 22:19	文件夹	
.gitignore	2026/8/14 12:33	Git Ignore 源文件	1 KB
file01.txt	2026/8/14 22:03	文本文档	0 KB
file02.txt	2026/8/14 22:07	文本文档	0 KB

```
changezhangmeng MINGW64 /d/desktop/git/test01 (mas
$ git-log
* 8f3c1a6 (HEAD -> master) update file01 count-3
| * 50f83f6 (dev) update file01 count-2
|/
|/
| * cf933d4 Merge branch 'dev01'
|/
| * ce5ab1d add file02
| * cbec514 add ignore file
```

分支岔开表示有多个分支在修改同一个文件

3.4.7、删除分支

不能删除当前分支，只能删除其他分支

- `git branch -d b1` 删除分支时，需要做各种检查
- `git branch -D b1` 不做任何检查，强制删除

小d删除了就使用D，一般使用d就够了

我们去删除没有合并的分支的时候就会出现删除不了的情况，此时就可以使用D

3.4.8、解决冲突

当我们合并分支后，两个或者多个分支对同一个文件的同一个地方进行修改的时候（不是同一个地方是不会出现冲突的），此时git就不知道要取哪个分支修改的值，是取a分支修改的值，还是取b分支修改的值呢，此时就产生了冲突

冲突的报错

```
zhang@zhangmeng MINGW64 ~/Desktop/git-test01 (master)
$ git merge dev
Auto-merging file01.txt
CONFLICT (content): Merge conflict in file01.txt
Automatic merge failed; fix conflicts and then commit the result.
```

此时的文件样子

```
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<<<<<<< HEAD
update count=3
=====
update count=2
>>>>>> dev
```

第一个count值表示的是当前分支修改的值

第二个count值是在dev分支修改的值

当两个分支上对文件的修改可能会存在冲突，例如同时修改了同一个文件的同一行，这时就需要手动解

决冲突，**解决冲突步骤如下：**

其实我们就是直接手动去删除文件中的一个分支，留下一个分支，这样就不会冲突了

1. 处理文件中冲突的地方
2. 将解决完冲突的文件加入暂存区(add)
3. 提交到仓库(commit)

冲突部分的内容处理如下所示：

```

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master)
$ git merge dev
Auto-merging file01.txt
CONFLICT (content): Merge conflict in file01.txt 提示file01.txt文件中有冲突
Automatic merge failed; fix conflicts and then commit the result.

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ cat file01.txt
<<<<<<< HEAD
count=1 <———— master分支上修改的内容
=====
count=2 <———— dev分支上修改的内容
>>>>>>> dev

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ vi file01.txt

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ cat file01.txt
count=2 <———— 修改成我们希望合并后的样子

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ git add . 添加到暂存区

zhang@zhangmeng MINGW64 /c/czbk_advanced/git-test (master|MERGING)
$ git commit 提交到本地仓库
[master 98eb214] Merge branch 'dev'

```

3.4.9、开发中分支使用原则与流程

几乎所有的版本控制系统都以某种形式支持分支。使用分支意味着你可以把你的工作从开发主线上分离

开来进行重大的Bug修改、开发新的功能，以免影响开发主线。

在开发中，一般有如下分支使用原则与流程：

- master（生产）分支

线上分支，主分支，中小规模项目作为线上运行的应用对应的分支；

- develop（开发）分支

是从master创建的分支，一般作为开发部门的主要开发分支，如果没有其他并行开发不同期上线

要求，都可以在此版本进行开发，阶段开发完成后，需要是合并到master分支,准备上线。

例如我们要开发新功能，我们要可以在develop分支上在建一个分支，新功能一般叫做feature分支，开发完以后在合并到 develop分支上面去，而不是直接提交到master分支，最后项目做完了develop在合并到master分支上

develop和master分支是不可删除的

- feature/xxxx分支（用完可删）

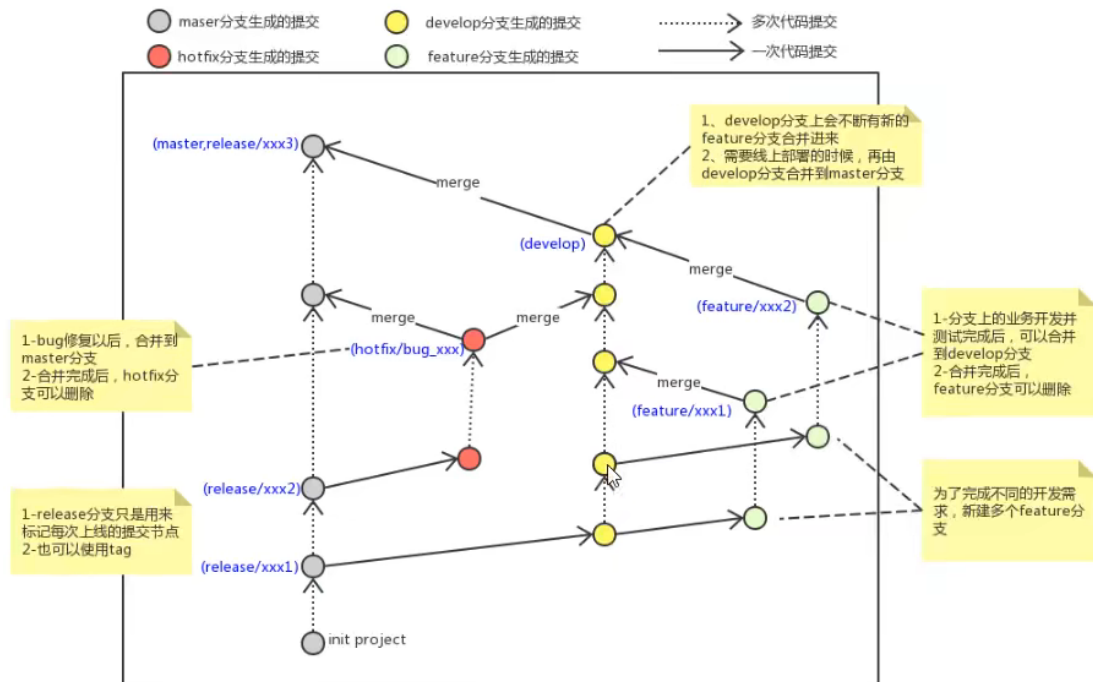
从develop创建的分支，一般是同期并行开发，但不同期上线时创建的分支，分支上的研发任务完

成后合并到develop分支，用完后可删除。

- hotfix/xxxx分支，

从master派生的分支，一般作为线上bug修复使用，修复测试完成后需要合并到master、test、develop分支。

- 还有一些其他分支，在此不再详述，例如test分支（用于代码测试）、pre分支（预上线分支）等等。



练习：分支操作

练习:分支操作

```
#####创建并切换到dev01分支，在dev01分支提交
# [master]创建分支dev01
git branch dev01
# [master]切换到dev01
git checkout dev01
# [dev01]创建文件file02.txt
略
# [dev01]将修改加入暂存区并提交到仓库,提交记录内容为: add file02 on dev
git add .
git commit -m 'add file02 on dev'
# [dev01]以精简的方式显示提交记录
git-log
#####切换到master分支，将dev01合并到master分支
# [dev01]切换到master分支
git checkout master
# [master]合并dev01到master分支
git merge dev01
# [master]以精简的方式显示提交记录
git-log
# [master]查看文件变化(目录下也出现了file02.txt)
略
#####删除dev01分支
# [master]删除dev01分支
git branch -d dev01
# [master]以精简的方式显示提交记录
git-log
```

4、Git远程仓库

4.1、常用的托管服务[远程仓库]

前面我们已经知道了Git中存在两种类型的仓库，即本地仓库和远程仓库。那么我们如何搭建Git远程仓库呢？我们可以借助互联网上提供的一些代码托管服务来实现，其中比较常用的有GitHub、码云、GitLab等。

github（地址：<https://github.com/>）是一个面向开源及私有软件项目的托管平台，因为只支持Git作为唯一的版本库格式进行托管，故名github

码云（地址：<https://gitee.com/>）是国内的一个代码托管平台，由于服务器在国内，所以相比于GitHub，码云速度会更快

GitLab（地址：<https://about.gitlab.com/>）是一个用于仓库管理系统的开源项目，使用Git作为代码管理工具，并在此基础上搭建起来的web服务，一般用于在企业、学校等内部网络搭建git私服。

企业里面我们一般使用GitLab,毕竟代码放在自己的机房里面才安全，gitLab要自己部署

4.2、注册码云

要想使用码云的相关服务，需要注册账号（地址：<https://gitee.com/signup>）

注册

已有帐号？[点此登录](#)

姓名

<https://gitee.com/>

个人空间地址



手机或邮箱

密码不少于6位



☐ 已阅读并同意 [使用条款](#) 以及 [非活跃帐号处理规范](#)

立即注册



使用OSChina帐号登录

其他方式登录



4.3、创建远程仓库

The screenshot shows the '新建仓库' (New Repository) page on Gitee. The form includes fields for repository name, path, and introduction. A dropdown menu is open for the '仓库名称' field, showing options like '新建仓库', '发布代码片段', '创建组织', '开通企业版', and '从 GitHub / GitLab 导入仓库'. The '仓库名称' field contains 'git_test'. The '路径' field contains 'git_test'. The '仓库地址' is 'https://gitee.com/czbk_zhang_meng/git_test'. The '仓库介绍' field contains 'git学习测试仓库'. The '是否开源' section has '私有' selected. The '选择语言' section has '请选择语言' selected. The '添加 .gitignore' section has '请选择 .gitignore 模板' selected. The '使用Readme文件初始化这个仓库' checkbox is checked. The '使用Issue模板文件初始化这个仓库' checkbox is checked. The '使用Pull Request模板文件初始化这个仓库' checkbox is checked. The '选择分支模型' dropdown is set to '单分支模型 (只创建 master 分支)'. The '创建' button is highlighted.

1-点击新建仓库

2-输入仓库地址

3-路径会自动生成

3-输入该仓库的用途

4-私有表示只有自己能看到，公开这所有人都能看到

5-默认即可，不要选择

6-点击创建

仓库创建完成后可以看到仓库地址，如下图所示：

The screenshot shows the repository page for '张猛 / git_test'. The 'SSH' tab is selected. The '快速设置' section shows the SSH address 'git@gitee.com:czbk_zhang_meng/git_test.g'. The 'HTTPS' tab is also visible. The 'SSH' tab is highlighted with a red box.

1-选择ssh

2-这里就是远程仓库的地址

4.4、配置SSH公钥

- 生成SSH公钥
 - ssh-keygen -t rsa
 - 不断回车

- 如果公钥已经存在，则自动覆盖
- Gitee设置账户公钥
 - 获取公钥
 - `cat ~/.ssh/id_rsa.pub`



- 验证是否配置成功
 - `ssh -T git@gitee.com`

4.5、操作远程仓库

4.5.1、添加远程仓库

此操作是先初始化本地库，然后与已创建的远程库进行对接。

我们要将本地仓库和远程仓库连接起来，一般一个本地仓库对应一个远程仓库远侧，远程仓库默认名为origin

- 命令：`git remote add <远端名称> <仓库路径SSH>`
 - 远端名称，默认是origin，取决于远端服务器设置
 - 仓库路径，从远端服务器获取此SSH，



- 例如: `git remote add origin git@gitee.com:icmc/mlcode.git`

```
xy@DESKTOP-JI3RDCR MINGW64 /e/postgraduate/machinelearning/program (master)
$ git remote add origin git@gitee.com:icmc/mlcode.git
```

4.5.2、查看远程仓库

- 命令：git remote

```
lxy@DESKTOP-JI3RDCR MINGW64 /e/postgraduate/machinelearning/program (master)
$ git remote
origin
```

4.5.3、推送到远程仓库

- 命令：git push [-f] [--set-upstream] [远端名称 [本地分支名][:远端分支名]]

- 如果远程分支名和本地分支名称相同，则可以只写本地分支

本来是：git push origin master : master 表示将本地仓库的master分支提交到远程仓库的master分支

- git push origin master 这里表示将本地仓库当前master分支的内容推到远程仓库上面去
- -f 表示强制覆盖
- --set-upstream 推送到远端的同时并且建立起和远端分支的关联关系。
 - git push --set-upstream origin master
- 如果当前分支已经和远端分支关联，则可以省略分支名和远端名。

```
zhang@zhangmeng MINGW64 ~/Desktop/git-test01 (master)
$ git push --set-upstream origin master:master
Everything up-to-date
Branch 'master' set up to track remote branch 'master' from 'origin'.

zhang@zhangmeng MINGW64 ~/Desktop/git-test01 (master)
$ git branch -vv
dev e96d2bc add file03
* master e96d2bc [origin/master] add file03

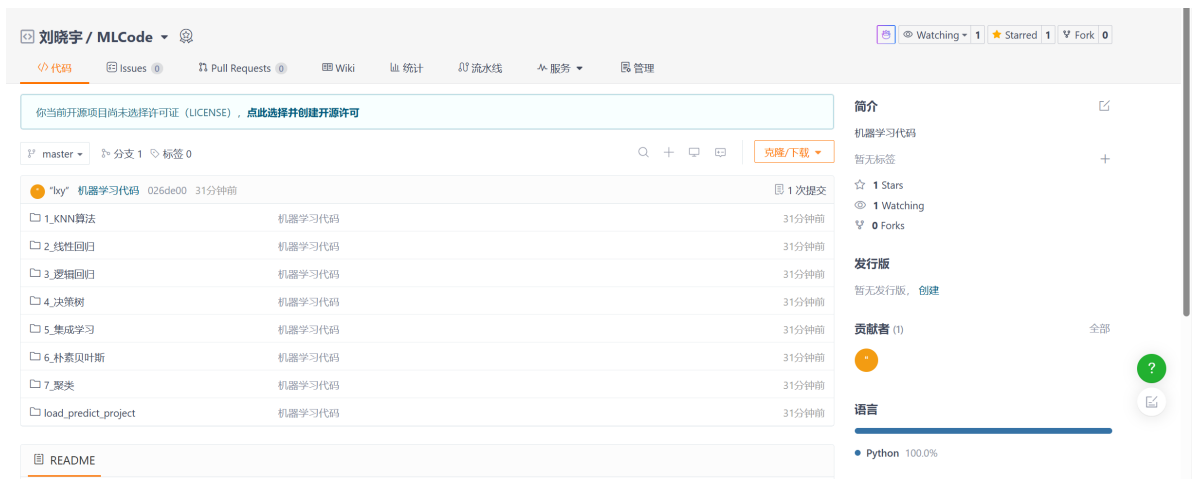
zhang@zhangmeng MINGW64 ~/Desktop/git-test01 (master)
$ git push
Everything up-to-date
```

将本地的master分支和远程的仓库的master分支绑定，之后提交使用git push

- git push 将master分支推送到已关联的远端分支。
- [-f] 表示强制覆盖远程仓库的内容

```
zhang@zhangmeng MINGW64 ~/Desktop/git-test (master)
$ git push --set-upstream origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 204 bytes | 204.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
remote: Powered by GITEE.COM [GNK-5.0]
To gitee.com:czbk_zhang_meng/git_test.git
* [new branch] master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

查询远程仓库



4.5.4、本地分支与远程分支的关联关系

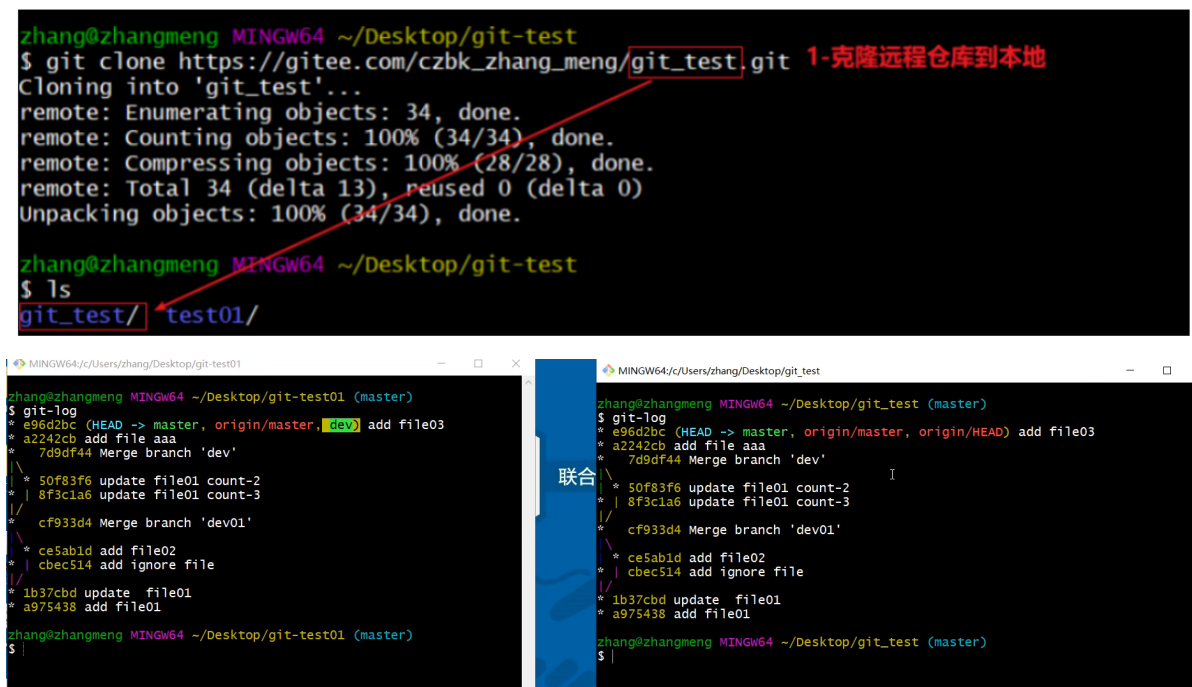
- 查看关联关系我们可以使用 `git branch -vv` 命令

```
lxy@DESKTOP-JI3RDCR MINGW64 /e/postgraduate/machinelearning/program (master)
$ git branch -vv
* master 026de00 机器学习代码
```

4.5.5、从远程仓库克隆

如果已经有一个远端仓库，我们可以直接clone到本地。

- 命令: `git clone <仓库路径> [本地目录]`
 - 本地目录可以省略，会自动生成一个目录，就是SSH后面那部分



左边是我们上传的仓库，右边是克隆下来的仓库

不同的主机都把修改完了版本资源放在远程仓库上，然后其他人都是克隆，这样就可以实现不同主机之间的数据同步了，数据都是一样的

克隆一般只会执行一次，就是在你进去公司的时候，别人提交了以后，我们不需要去克隆整个仓库，仓库是很大的，克隆也需要花很多时间，所以要去远程仓库中抓取我们要的版本信息，就是那些别人新增加的提交

4.5.6、从远程仓库中抓取和拉取

远程分支和本地的分支一样，我们可以进行merge操作，只是需要先把远端仓库里的更新都下载到本地，再进行操作。

- 抓取 命令：git fetch [remote name] [branch name]
 - 抓取指令就是将仓库里的更新都抓取到本地，不会进行合并，不合并本地仓库就是没有更新，此时还没有拿到远程仓库的内容，合并后才会拿到更新的内容
 - 如果不指定远端名称和分支名，则抓取所有分支。
- 拉取 命令：git pull [remote name] [branch name]
 - 拉取指令就是将远端仓库的修改拉到本地并自动进行合并，等同于fetch+merge
 - 如果不指定远端名称和分支名，则抓取所有并更新当前分支。

只去执行抓取命令 git fetch，本地仓库是没有远程仓库来的内容的

```
86183@LAPTOP-7FDEMLUI MINGW64 ~/Desktop/test1 (master)
$ git-log
* 12404aa (origin/master, origin/HEAD) create file test06
* 9e9e85d (HEAD -> master) create file test05
* ac98fb5 create file test4
* fc26c23 create file test03
* 6959a17 create file test02
* 08ae283 create file test01
* 0294409 修改了文件test
* f1291da 新建文件test
```

此电脑 > OS (C:) > 用户 > 86183 > 桌面 > test1

名称	修改日期	类型	大小
.git	2022/5/4 21:25	文件夹	
.gitignore	2022/5/4 21:03	Text Document	0 KB
test.txt	2022/5/4 21:03	文本文档	1 KB
test01.txt	2022/5/4 21:03	文本文档	0 KB
test02.txt	2022/5/4 21:03	文本文档	0 KB
test03.txt	2022/5/4 21:03	文本文档	0 KB
test04.txt	2022/5/4 21:03	文本文档	0 KB
test05.txt	2022/5/4 21:03	文本文档	0 KB

此时的test1中没有test06

将origin/master合并到本地仓库的master


```

86183@LAPTOP-7FDEMLUI MINGW64 ~/Desktop/test1 (master)
$ git merge origin/master
Updating 9e9e85d..12404aa
Fast-forward
 test06.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test06.txt

86183@LAPTOP-7FDEMLUI MINGW64 ~/Desktop/test1 (master)
$ git-log
* 12404aa (HEAD -> master, origin/master, origin/HEAD) create file test06
* 9e9e85d create file test05
* ac98fb5 create file test4
* fc26c23 create file test03
* 6959a17 create file test02
* 08ae283 create file test01
* 0294409 修改了文件test
* f1291da 新建文件test

```

此时test1中就有test06文件了

此电脑 > OS (C:) > 用户 > 86183 > 桌面 > test1

名称	修改日期	类型	大小
.git	2022/5/4 21:30	文件夹	
.gitignore	2022/5/4 21:03	Text Document	0 KB
test.txt	2022/5/4 21:03	文本文档	1 KB
test01.txt	2022/5/4 21:03	文本文档	0 KB
test02.txt	2022/5/4 21:03	文本文档	0 KB
test03.txt	2022/5/4 21:03	文本文档	0 KB
test04.txt	2022/5/4 21:03	文本文档	0 KB
test05.txt	2022/5/4 21:03	文本文档	0 KB
test06.txt	2022/5/4 21:30	文本文档	0 KB

所以我们一般执行拉取命令

直接执行git pull

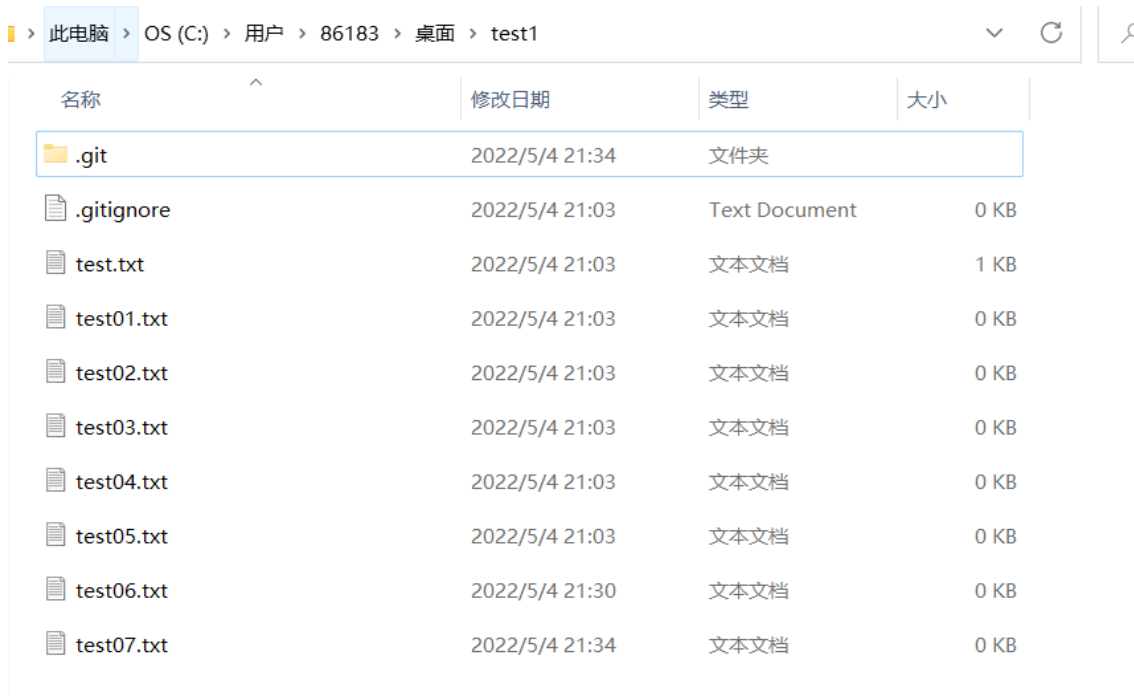
```

86183@LAPTOP-7FDEMLUI MINGW64 ~/Desktop/test1 (master)
$ git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (2/2), 208 bytes | 29.00 KiB/s, done.
From gitee.com:lingyi-study/test
 12404aa..2a0e6f9 master    -> origin/master
Updating 12404aa..2a0e6f9
Fast-forward
 test07.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test07.txt

86183@LAPTOP-7FDEMLUI MINGW64 ~/Desktop/test1 (master)
$ git-log
* 2a0e6f9 (HEAD -> master, origin/master, origin/HEAD) create file test07
* 12404aa create file test06
* 9e9e85d create file test05
* ac98fb5 create file test4
* fc26c23 create file test03
* 6959a17 create file test02
* 08ae283 create file test01
* 0294409 修改了文件test
* f1291da 新建文件test

```


此时test1中有直接拿到文件test07

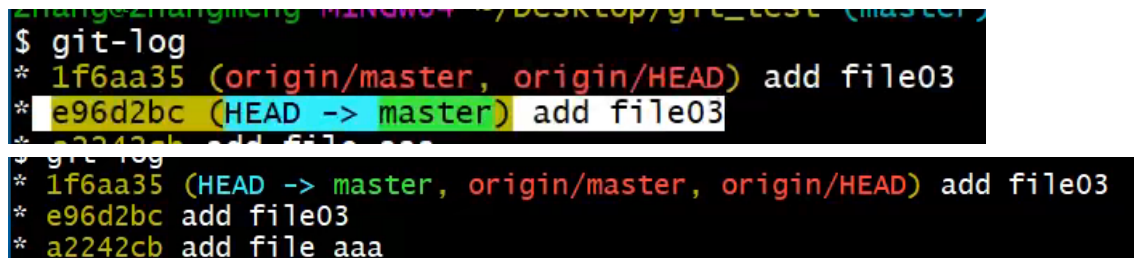


名称	修改日期	类型	大小
.git	2022/5/4 21:34	文件夹	
.gitignore	2022/5/4 21:03	Text Document	0 KB
test.txt	2022/5/4 21:03	文本文档	1 KB
test01.txt	2022/5/4 21:03	文本文档	0 KB
test02.txt	2022/5/4 21:03	文本文档	0 KB
test03.txt	2022/5/4 21:03	文本文档	0 KB
test04.txt	2022/5/4 21:03	文本文档	0 KB
test05.txt	2022/5/4 21:03	文本文档	0 KB
test06.txt	2022/5/4 21:30	文本文档	0 KB
test07.txt	2022/5/4 21:34	文本文档	0 KB

为什么将远程仓库更新的资源抓取到本地要合并呢？

总结：push的时候要使远程仓库的更新是最新的，就是最后一个修改的版本要使远程仓库的，pull要使本地仓库的更新是最新的，就是最后一个修改的版本要使本地仓库的

因为我们此时我们获取到的是远程仓库的版本更新，但是我们本地的版本不是最新的，也就是说此时我们本地和远程仓库不是同步的，所以我们要将远端拿到的修改合并到本地仓库的master上，使得本地的版本修改变为最新的



```
$ git-log
* 1f6aa35 (origin/master, origin/HEAD) add file03
* e96d2bc (HEAD -> master) add file03
* a2242cb add file aaa

$ git-log
* 1f6aa35 (HEAD -> master, origin/master, origin/HEAD) add file03
* e96d2bc add file03
* a2242cb add file aaa
```

发现此时本地的修改和远程仓库的修改同步了

1. 在test01这个本地仓库进行一次提交并推送到远程仓库

```

zhang@zhangmeng MINGW64 ~/Desktop/git-test/test01 (master)
$ touch file05.txt

zhang@zhangmeng MINGW64 ~/Desktop/git-test/test01 (master)
$ git add .

zhang@zhangmeng MINGW64 ~/Desktop/git-test/test01 (master)
$ git commit -m'创建file05'
[master b39fb8b] 创建file05
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file05.txt

zhang@zhangmeng MINGW64 ~/Desktop/git-test/test01 (master)
$ git push 02-推送到远程仓库
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 268 bytes | 268.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0)
remote: Powered by GITEE.COM [GNK-3.8]
To https://gitee.com/czbk_zhang_meng/git_test.git
664d35c..b39fb8b master -> master

zhang@zhangmeng MINGW64 ~/Desktop/git-test/test01 (master)
$

```

01-在test01这个仓库下进行一次提交

2. 在另一个仓库将远程提交的代码拉取到本地仓库

```

zhang@zhangmeng MINGW64 ~/Desktop/git-test/git_test (master)
$ git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 1), reused 0 (delta 0)
Unpacking objects: 100% (2/2), done.
From https://gitee.com/czbk_zhang_meng/git_test
664d35c..b39fb8b master -> origin/master
Updating 664d35c..b39fb8b
Fast-forward
 file05.txt | 0
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file05.txt

```

1-将远程仓库的修改拉取到本地仓库

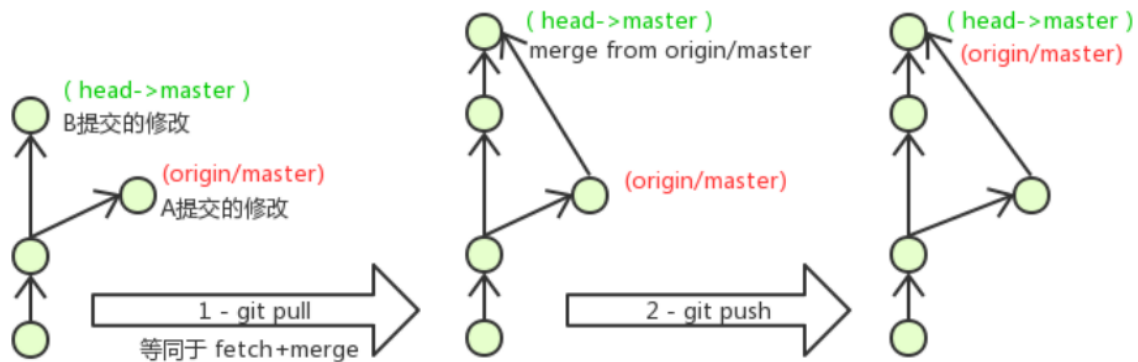
4.5.7、解决合并冲突

我们要更新远程仓库的资源时，先要获取此时远程仓库的资源后，在合并到自己的master分支中，然后再上传到远程仓库上

在一段时间，A、B用户修改了同一个文件，且修改了同一行位置的代码，此时会发生合并冲突。

A用户在本地修改代码后优先推送到远程仓库，此时B用户在本地修订代码，提交到本地仓库后，也需要

推送到远程仓库，此时B用户晚于A用户，故需要先拉取远程仓库的提交，经过合并后才能推送到远端分支，如下图所示。



在B用户拉取代码时，因为A、B用户同一段时间修改了同一个文件的相同位置代码，故会发生合并冲突。

突。

就是b在更新一个资源之前，有一个a在b之前率先改掉了这个资源，此时就会出现分支冲突的问题，git不知道是要取a修改的值，还是b修改的值，此时就要我们手动去对应文件里去修改，到底要保留哪一个

这不是一个先提交，一个后提交吗，拿到的前一个提交的版本信息，为什么还会冲突呢，为什么不能直接修改count的值呢，这个同一时间有点抽象，????????? 这里不理解

远程分支也是分支，所以合并时冲突的解决方式也和解决本地分支冲突相同相同，在此不再赘述，需要

学员自己练习。

练习:远程仓库操作

```
#####1-将本地仓库推送到远程仓库
# 完成4.1、4.2、4.3、4.4的操作
略
# [git_test01]添加远程仓库
git remote add origin git@gitee.com/**/**.git
# [git_test01]将master分支推送到远程仓库,并与远程仓库的master分支绑定关联关系
git push --set-upstream origin master

#####2-将远程仓库克隆到本地
# 将远程仓库克隆到本地git_test02目录下
git clone git@gitee.com/**/**.git git_test02
# [git_test02]以精简的方式显示提交记录
git-log

#####3-将本地修改推送到远程仓库
# [git_test01]创建文件file03.txt
略
# [git_test01]将修改加入暂存区并提交到仓库,提交记录内容为: add file03
git add .
git commit -m 'add file03'
# [git_test01]将master分支的修改推送到远程仓库
git push origin master

#####4-将远程仓库的修改更新到本地
# [git_test02]将远程仓库修改再拉取到本地
git pull
# 以精简的方式显示提交记录
git-log
# 查看文件变化(目录下也出现了file03.txt)
略
```

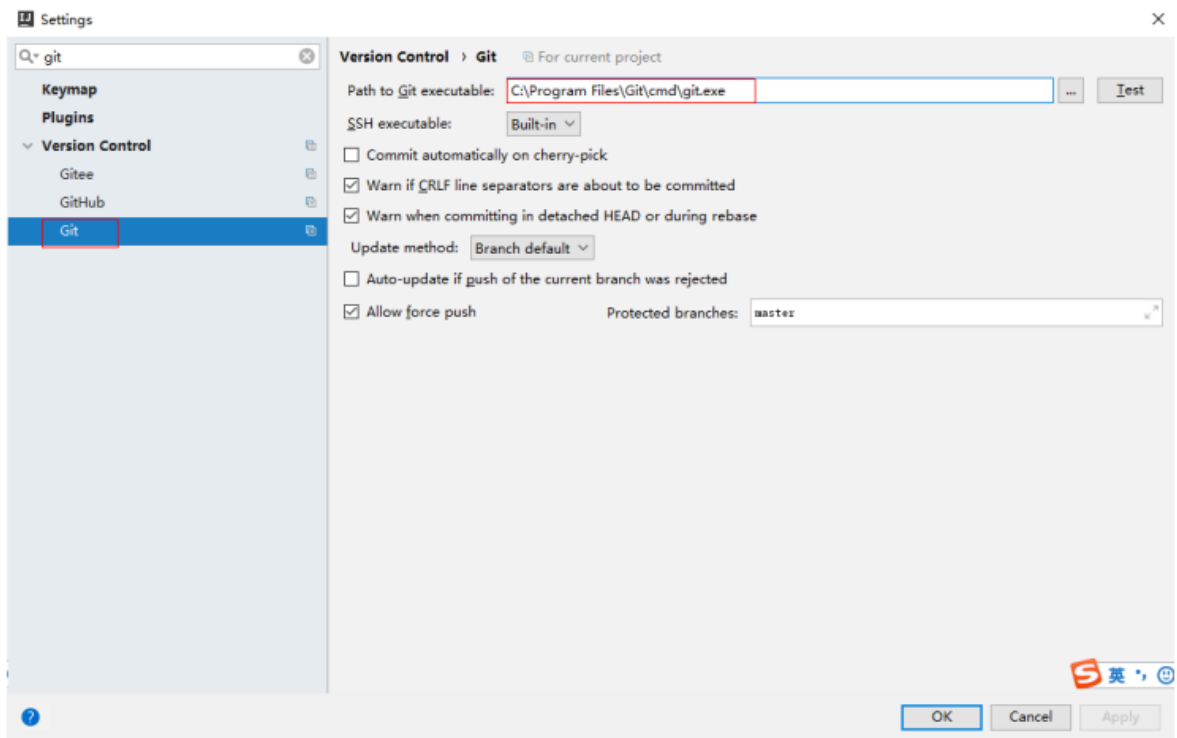
5、在Idea中使用Git（重点掌握）

5.1、在Idea中配置Git

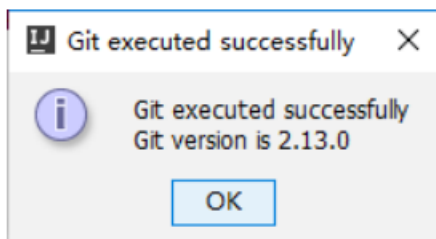
安装好IntelliJ IDEA后，如果Git安装在默认路径下，那么idea会自动找到git的位置，如果更改了Git的安

装位置则需要手动配置下Git的路径。选择File→Settings打开设置窗口，找到Version Control下的git选

项：



点击Test按钮,现在执行成功，配置完成



5.2、在Idea中操作Git

场景：本地已经有一个项目，但是并不是git项目，我们需要将这个放到码云的仓库里，和其他开发人员

继续一起协作开发。

5.2.1、创建项目远程仓库（参照4.3）

新建仓库

仓库名称 ✓

springmvc_demo

归属

张猛

路径

springmvc_demo

仓库地址: https://gitee.com/czbk_zhang_meng/springmvc_demo

仓库介绍 非必填

springMvc小demo, 用于git学习、linux项目发布练习

是否开源

☐ 私有 ☒ 公开

任何人都可以访问该仓库的代码和其他任何形式的资源

选择语言

请选择语言

添加 .gitignore

请选择 .gitignore 模板

添加开源许可证 ⓘ

请选择开源许可证

☐ 使用Readme文件初始化这个仓库

☐ 使用Issue模板文件初始化这个仓库 ⓘ

☐ 使用Pull Request模板文件初始化这个仓库 ⓘ

后续我们直接把本地仓库推送上来, 所以这里都不要勾选

选择分支模型 (仓库初始化后将根据所选分支模型创建分支)

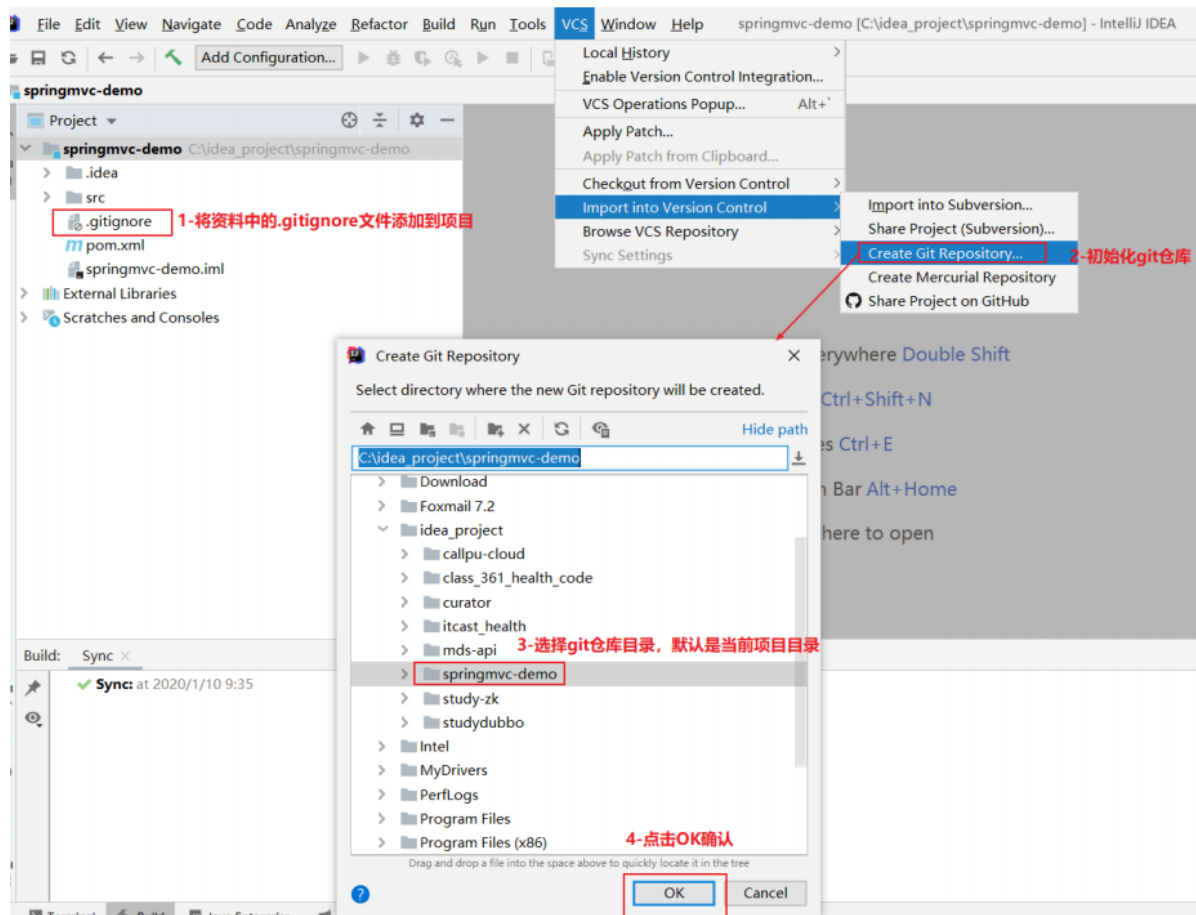
单分支模型 (只创建 master 分支)

导入已有仓库

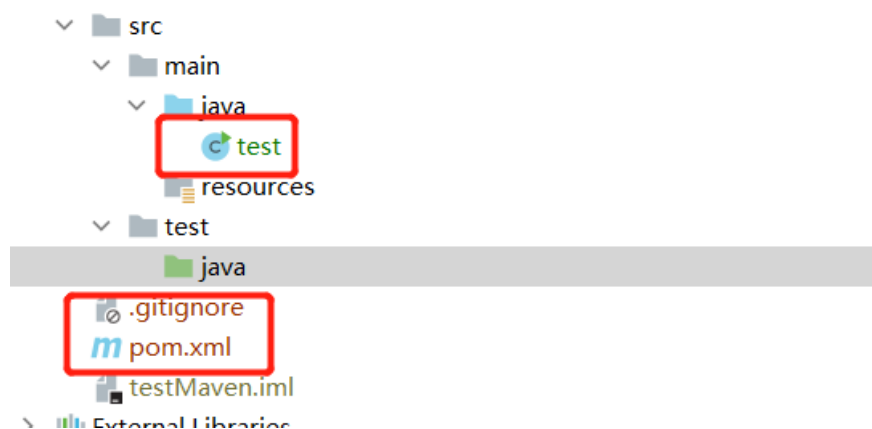
创建

5.2.2、初始化本地仓库 (将当前项目初始化为仓库)

在idea中创建文件的时候, 它会询问你是否要将文件添加到git中, 修改文件的时候, idea会自动帮我们去add, 我们只需要去commit就好了



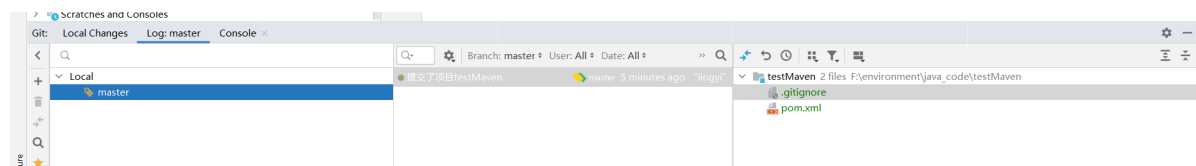
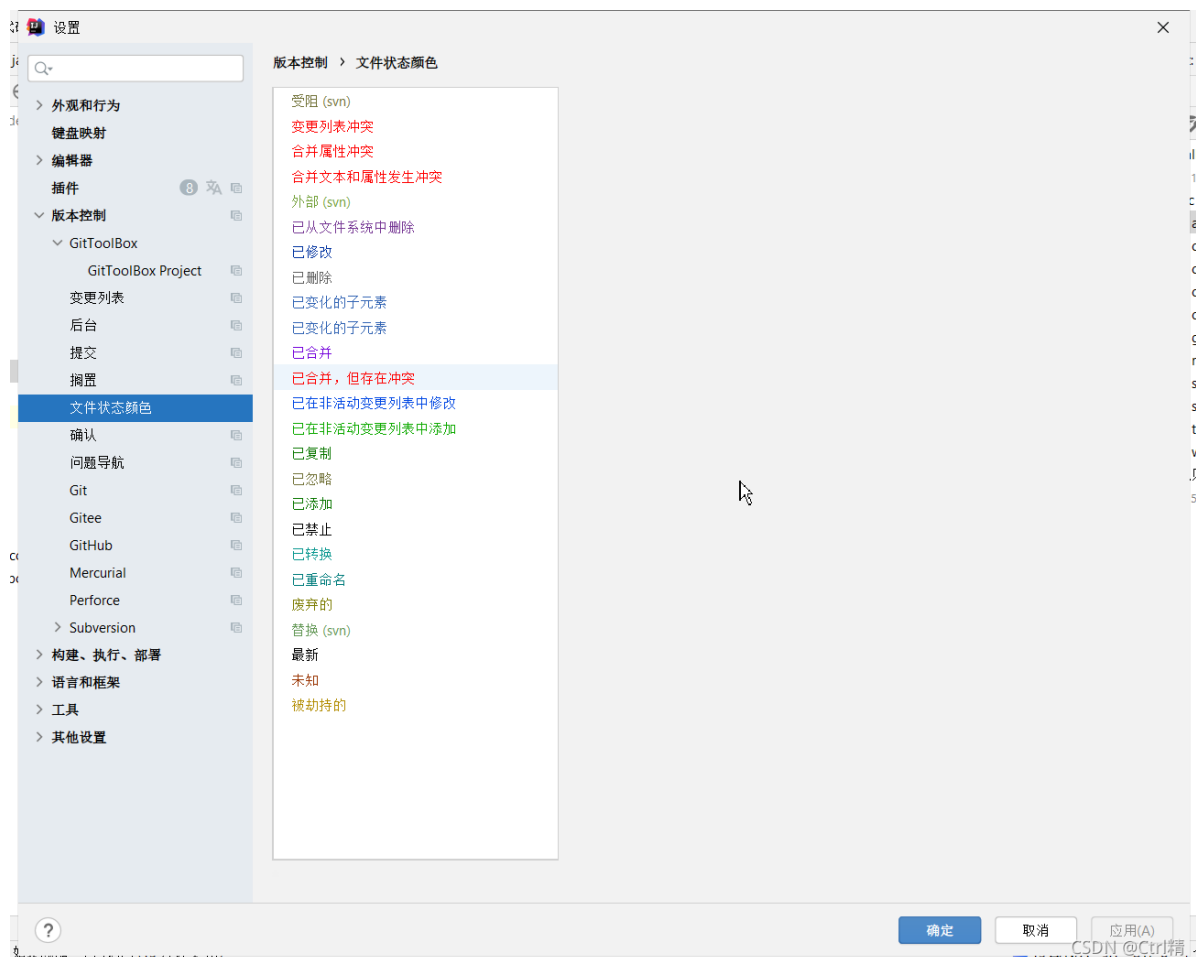
此时我们的项目目录就变成了一个本地仓库



绿色的文件代表已添加到git中

爆红的文件没有被添加到git当中，被Git识别为冲突文件

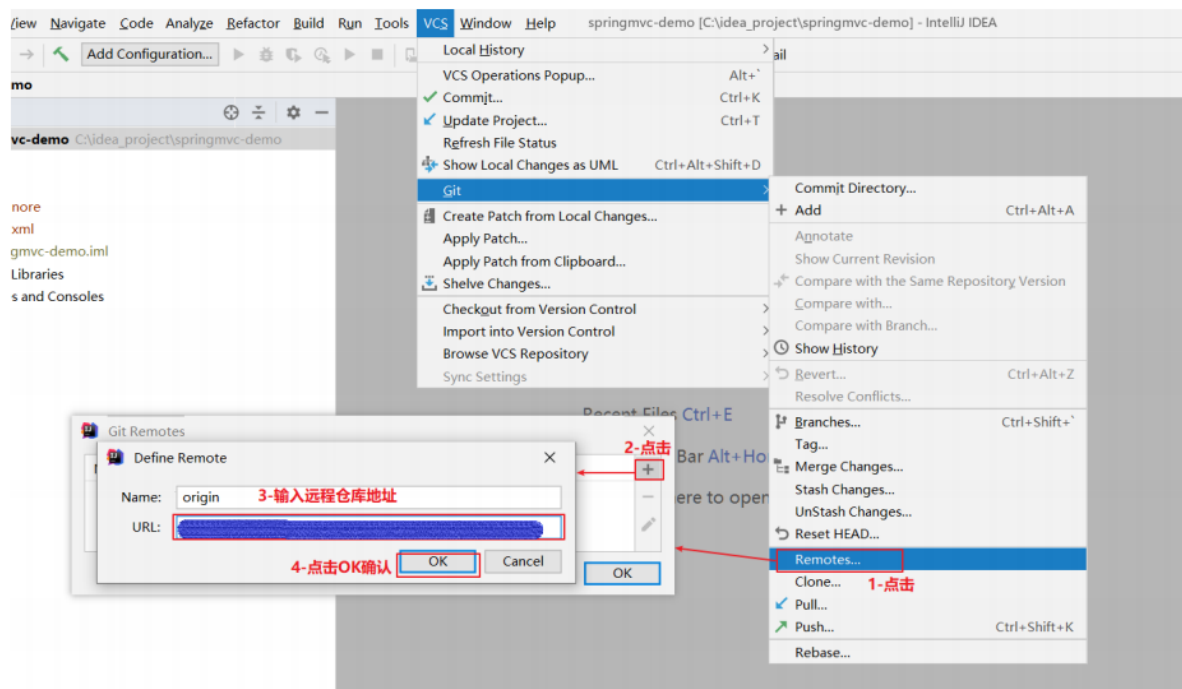
灰色的文件代表已忽略的文件，可以在gitignore文件中配置



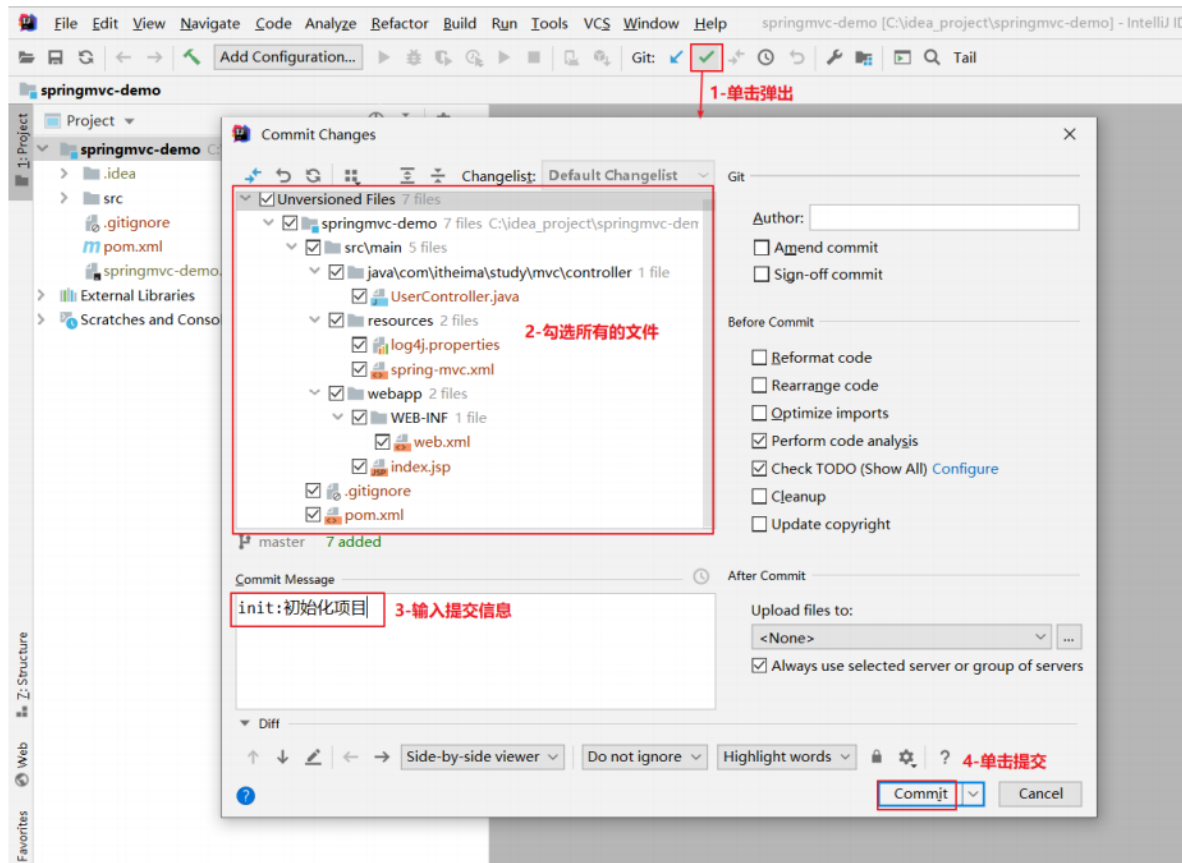
这里只是我们的本地仓库

5.2.3、设置远程仓库

远程仓库名默认为origin

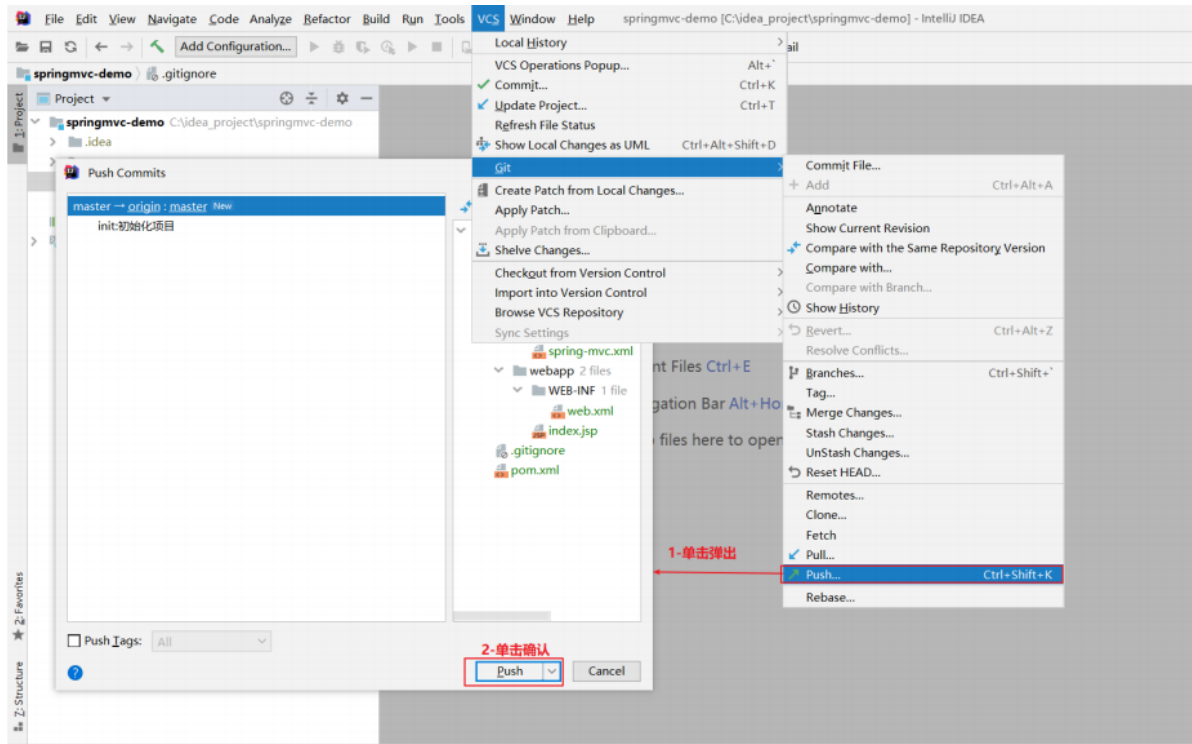


5.2.4、提交到本地仓库

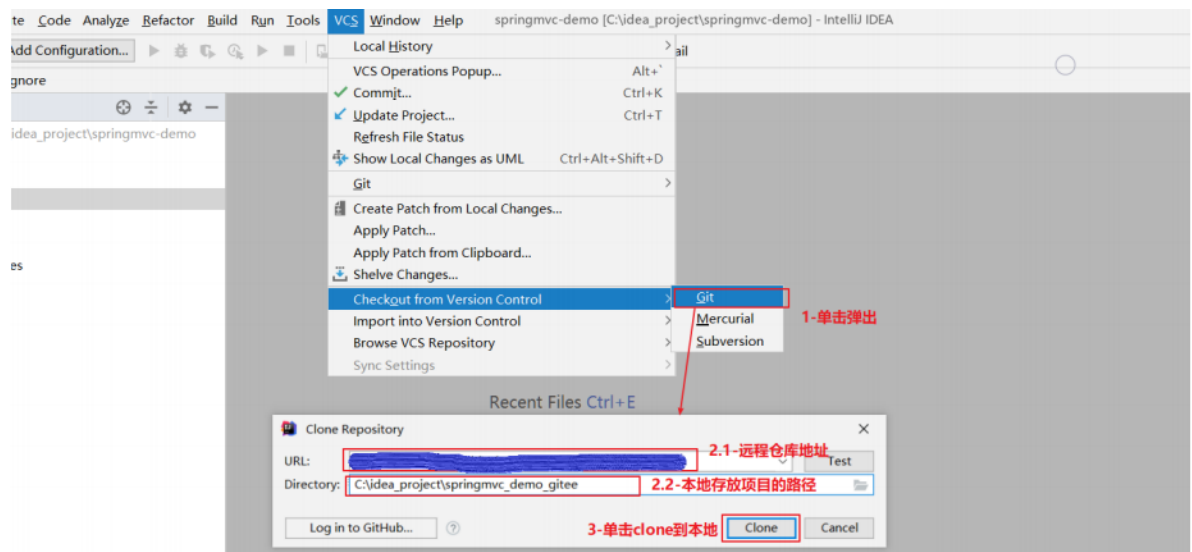


5.2.6、推送到远程仓库

在将本地仓库的修改推送到git远程仓库的时候，我们要先pull，先拿到此时远程仓库的版本信息，再去此版本信息上修改

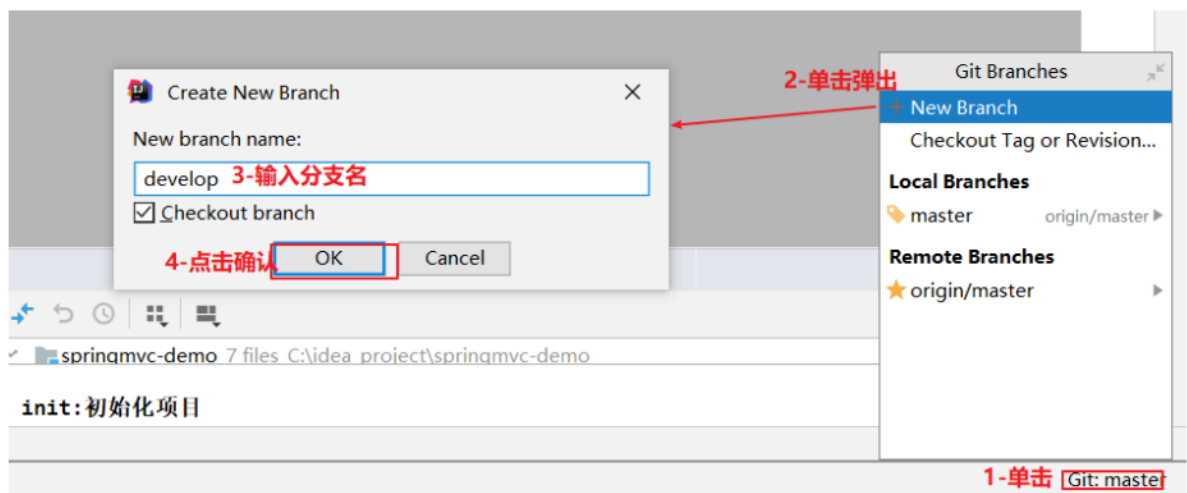


5.2.7、克隆远程仓库到本地

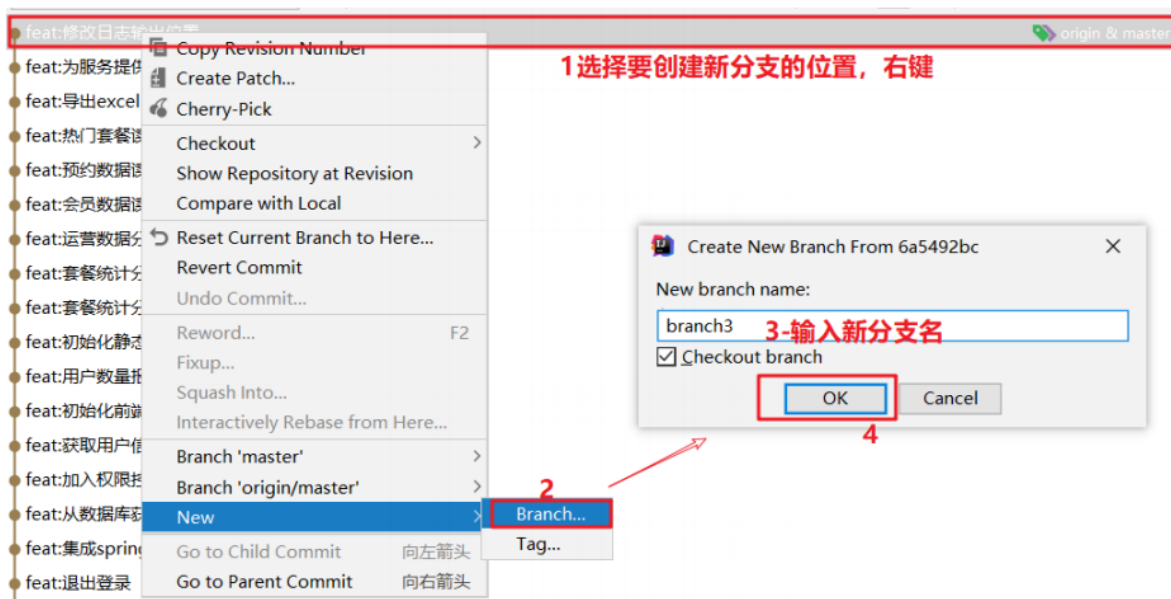


5.2.8、创建分支

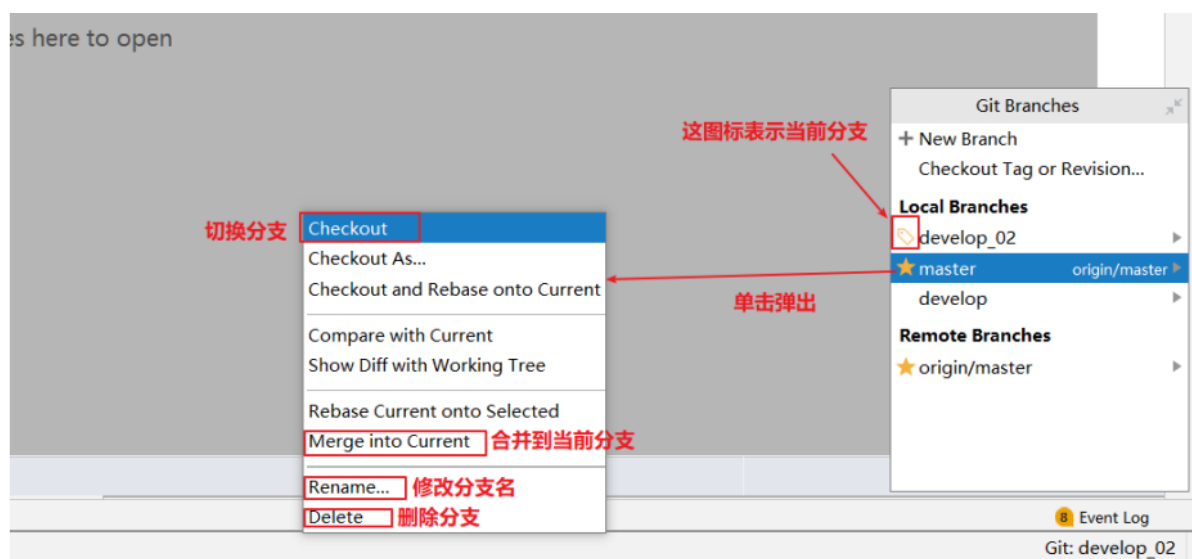
- 最常规的方式



- 最强大的方式

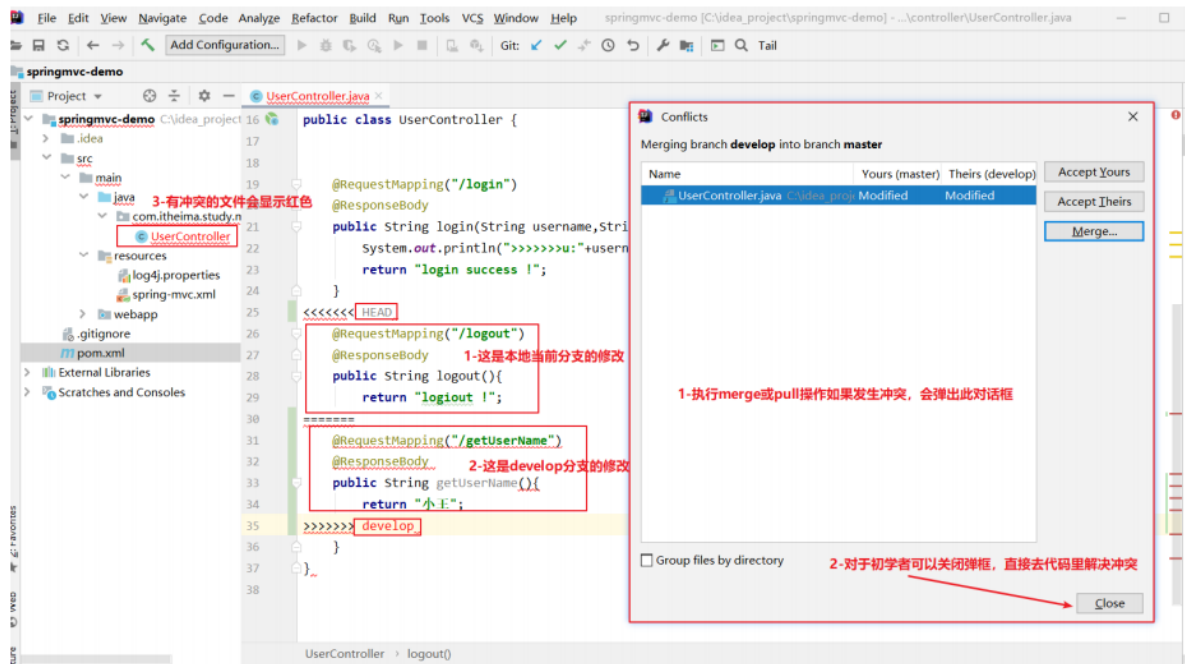


5.2.9、切换分支及其他分支相关操作

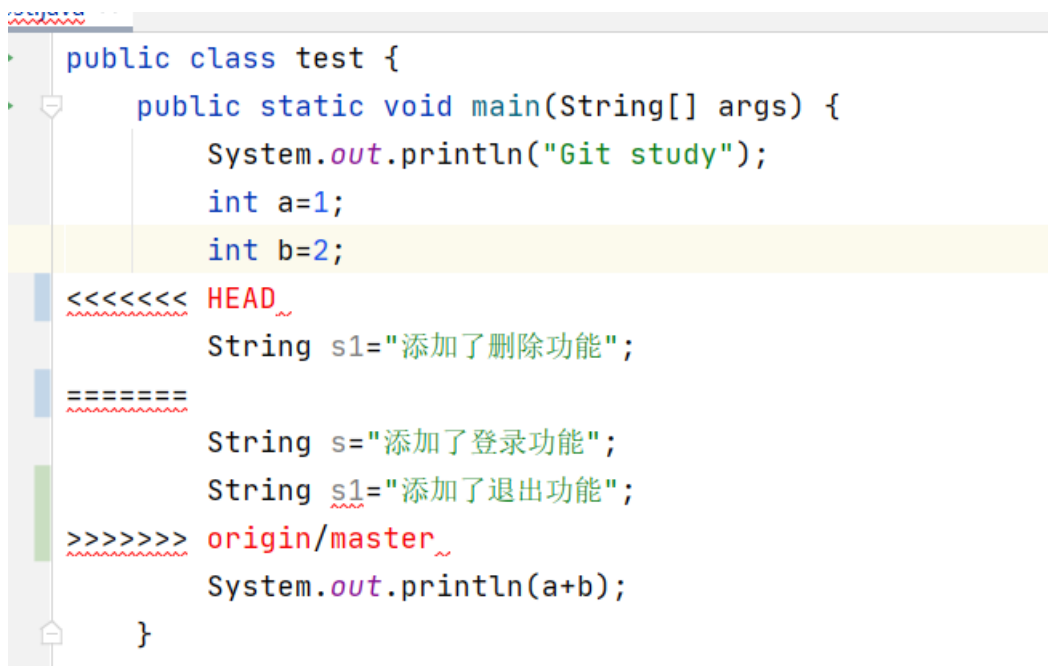


5.2.11、解决冲突

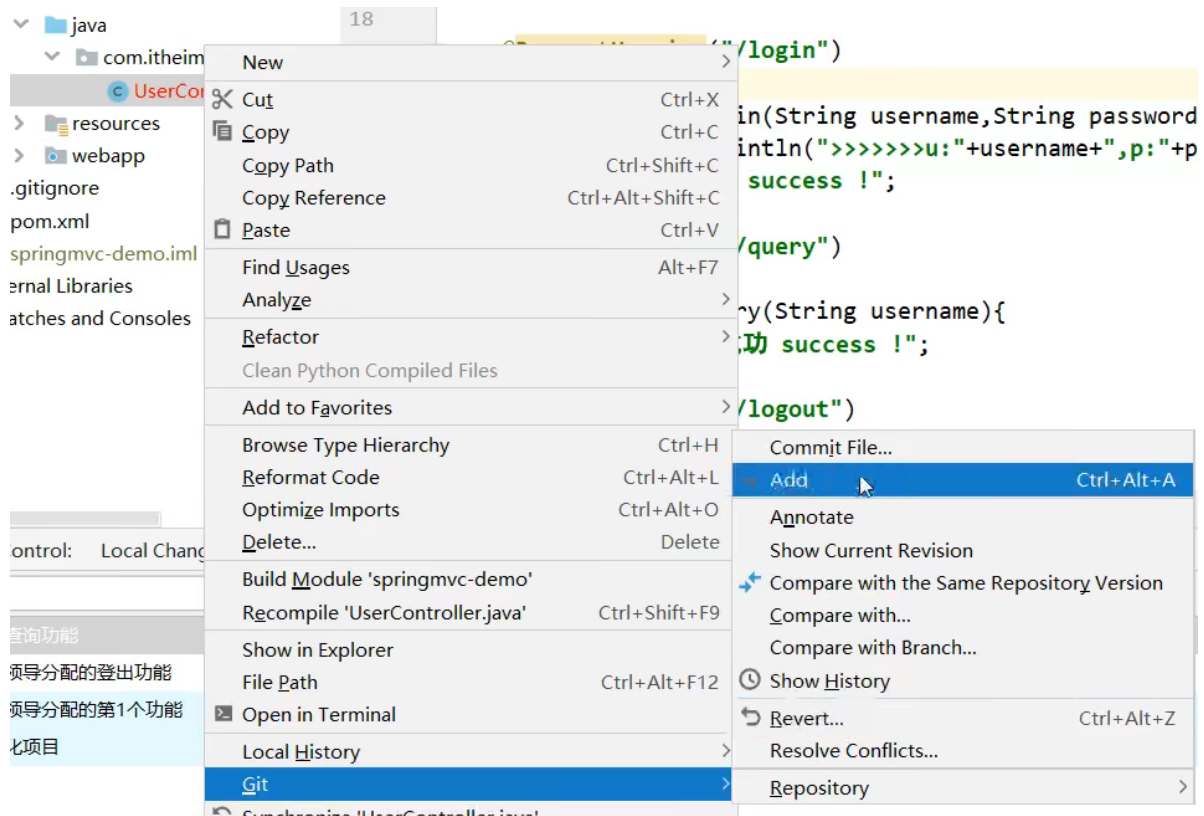
1. 执行merge或pull操作时，可能发生冲突



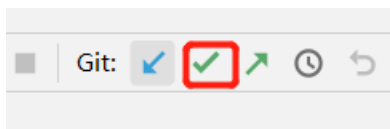
2. 解决文件里的冲突，删除提示信息



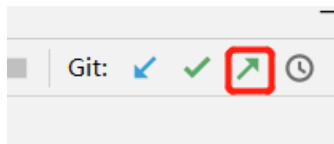
3. 冲突解决后加入暂存区



3. 提交到本地仓库

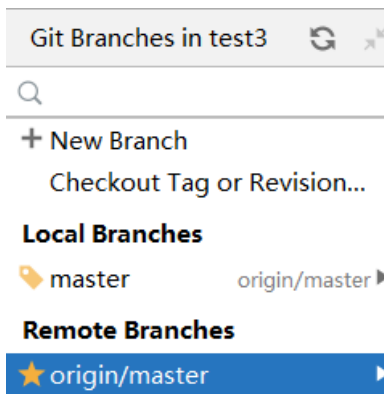


4. 推送到远程仓库



5.2.12查看分支

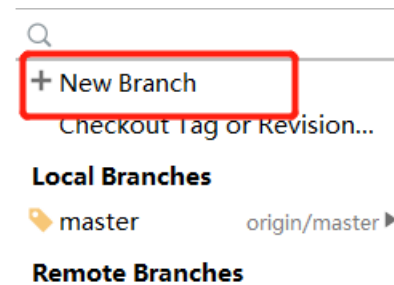




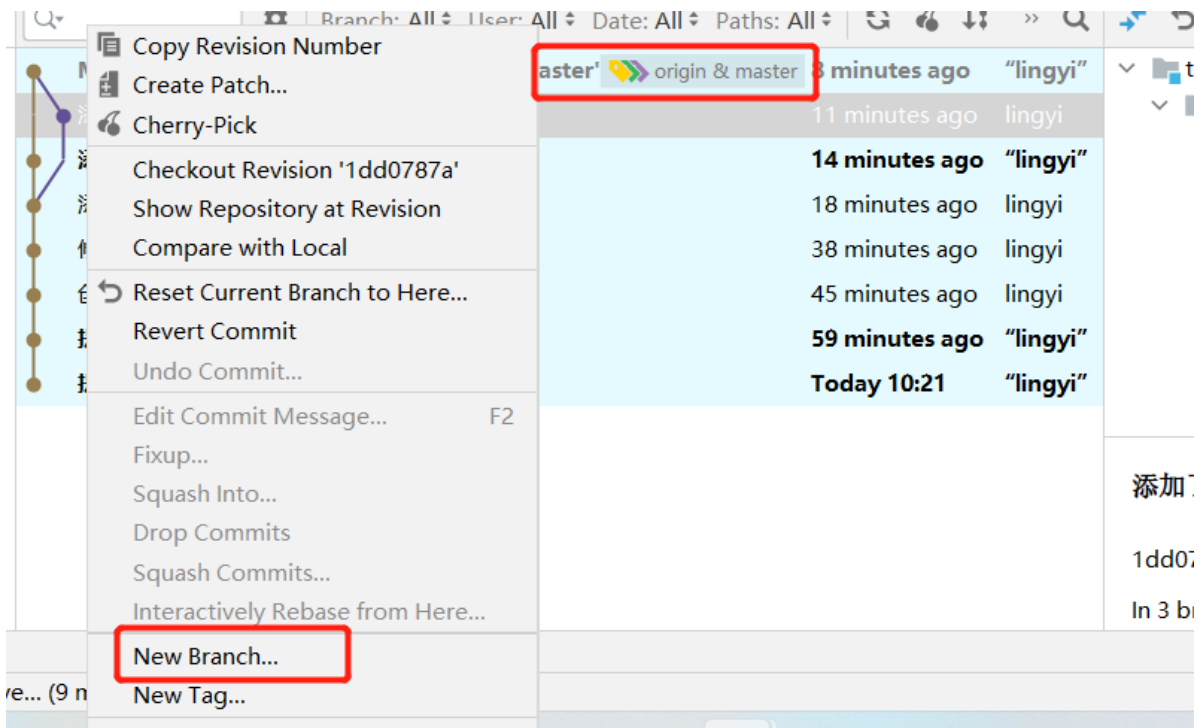
可以看到本地仓库的分支和远程仓库的分支

5.2.13创建分支

方式一：

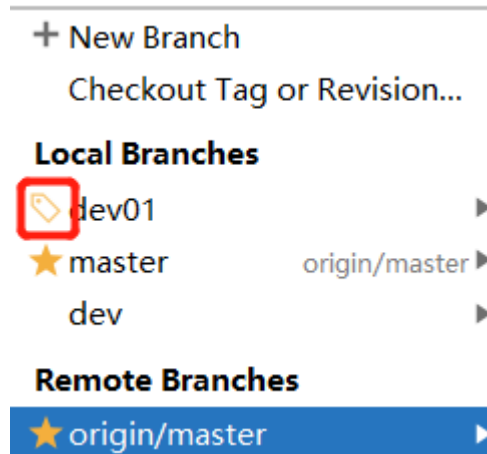


方式二：（推荐）



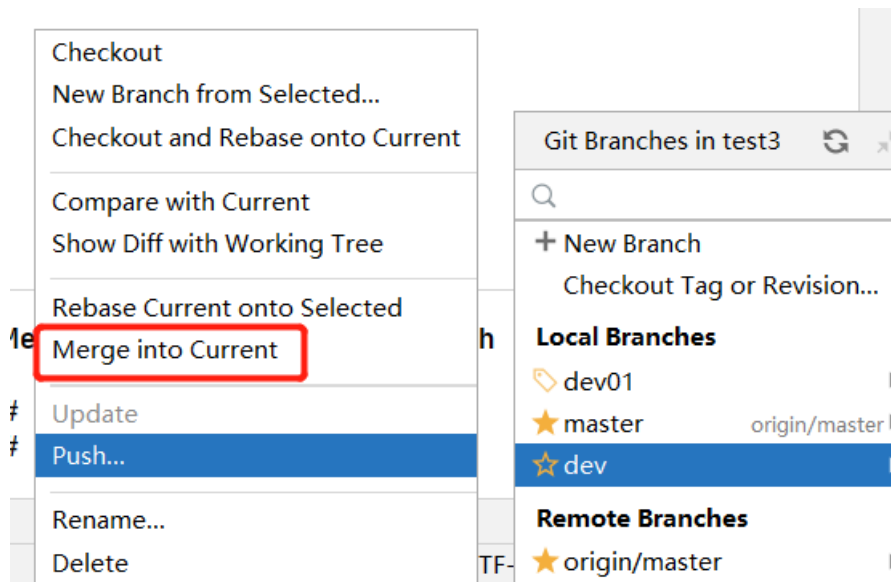
这种方式还可以看到分支所在的版本信息

5.2.14合并分支



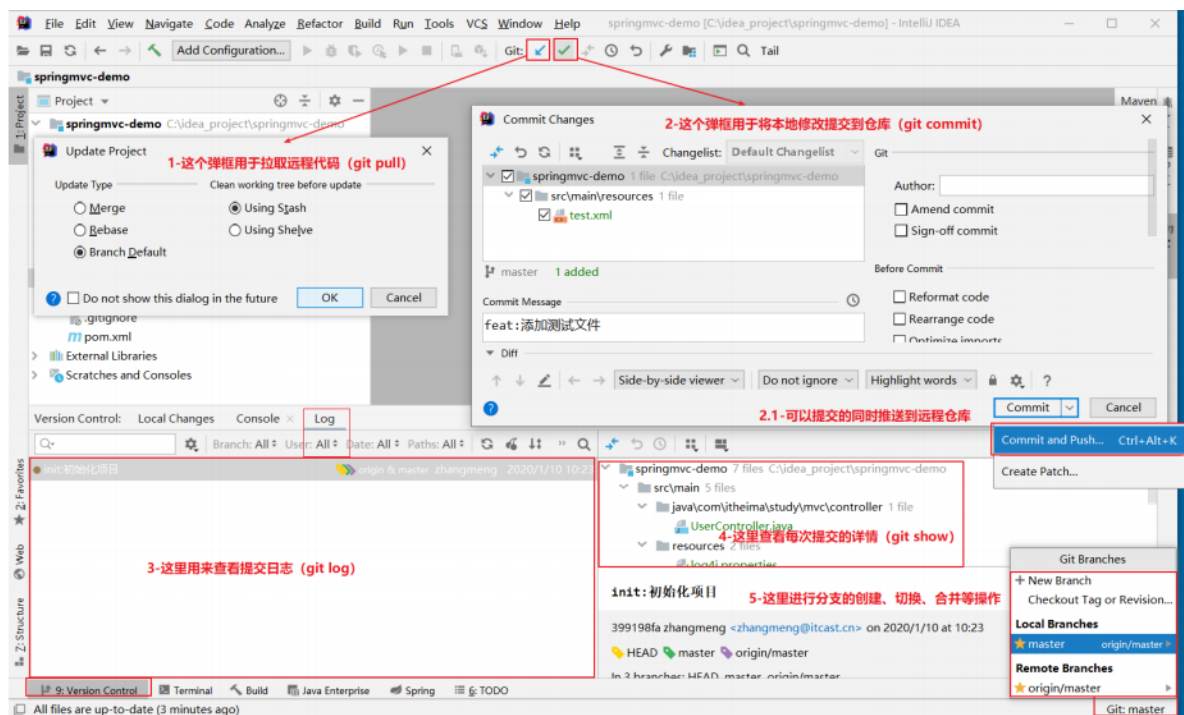
这个标签表示是当前分支

我们到想要合并的分支上面，右键

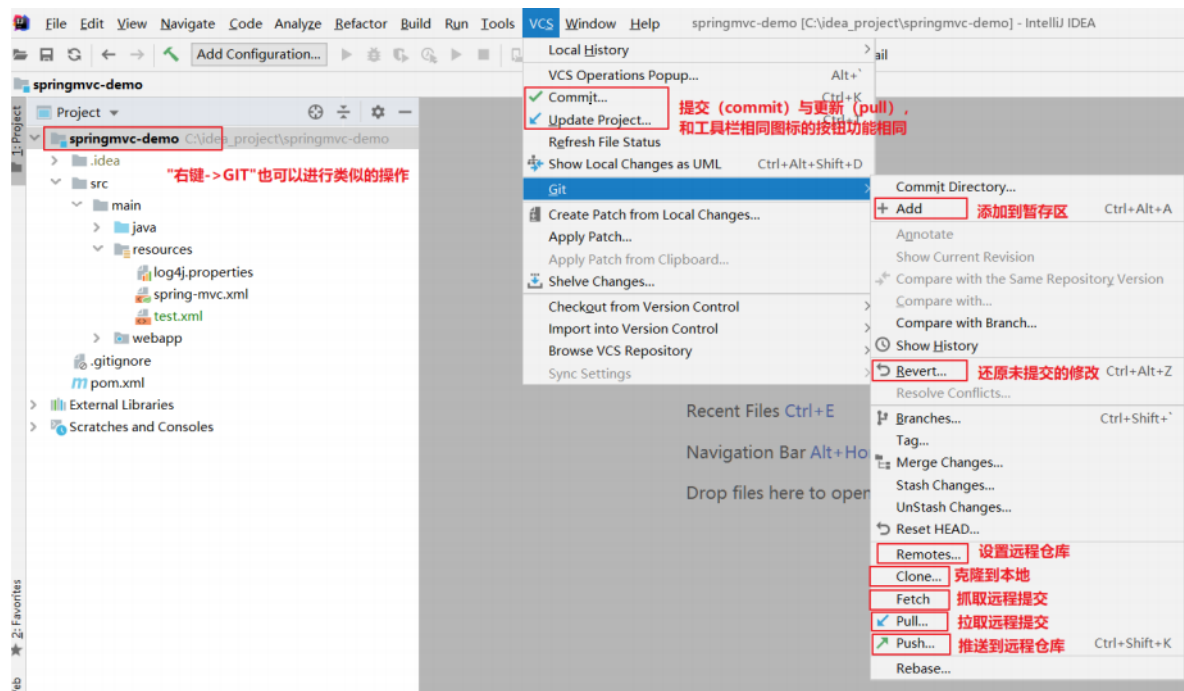


5.3、IDEA常用GIT操作入口

1. 第一张图上的快捷入口可以基本满足开发的需求。



2. 第二张图是更多在IDEA操作git的入口。



####

5.4、场景分析

基于我们后面的实战模式，我们做一个综合练习

当前的开发环境如下，我们每个人都对这个项目已经开发一段时间，接下来我们要切换成团队开发模

式。

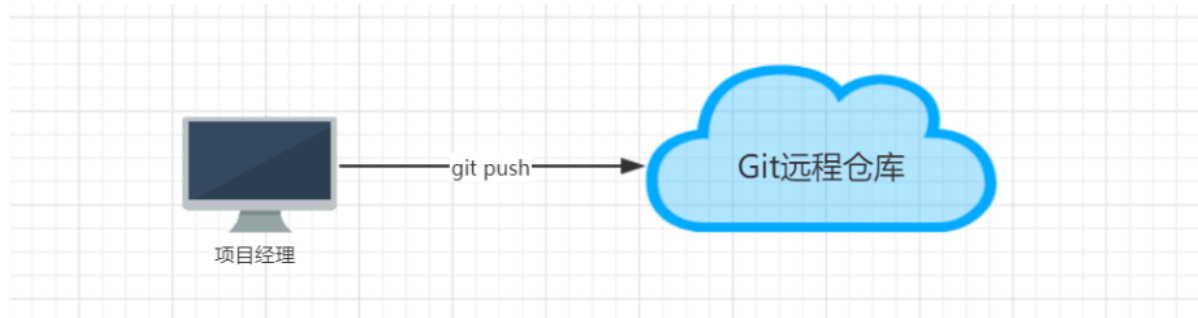
也就是我们由一个团队来完成这个项目实战的内容。团队有组长和若干组员组成（组长就是开发中的项

目经理）。

所有操作都在idea中完成。

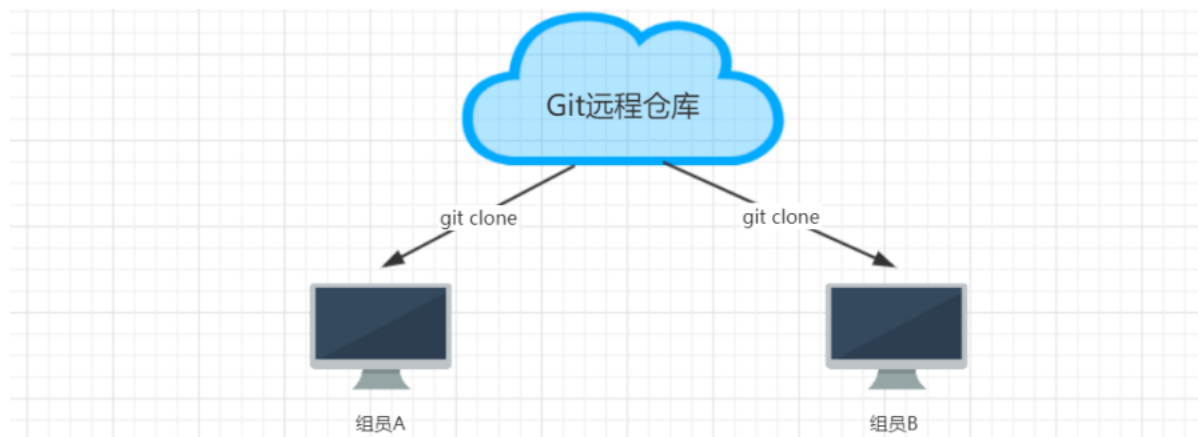
练习场景如下：

1、由组长，基于本项目创建本地仓库；创建远程仓库，推送项目到远程仓库。



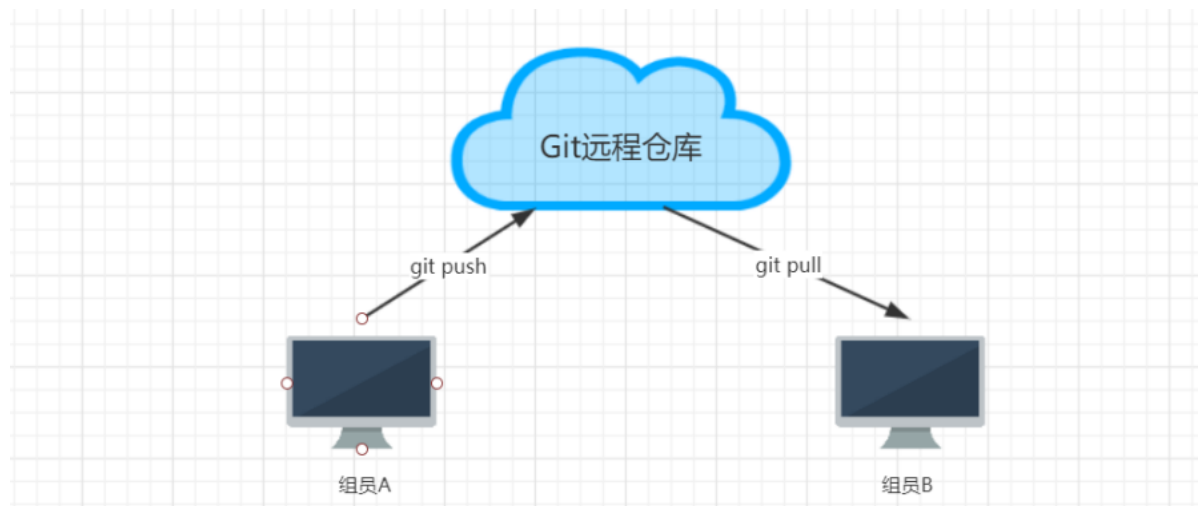
2、每一位组员从远程仓库克隆项目到idea中,这样每位同学在自己电脑上就有了一个工作副本，可以正

式的开始开发了。我们模拟两个组员(组员A、组员B)，克隆两个工作区。



3、组员A修改工作区,提交到本地仓库，再推送到远程仓库。组员B可以直接从远程仓库获取最新的代

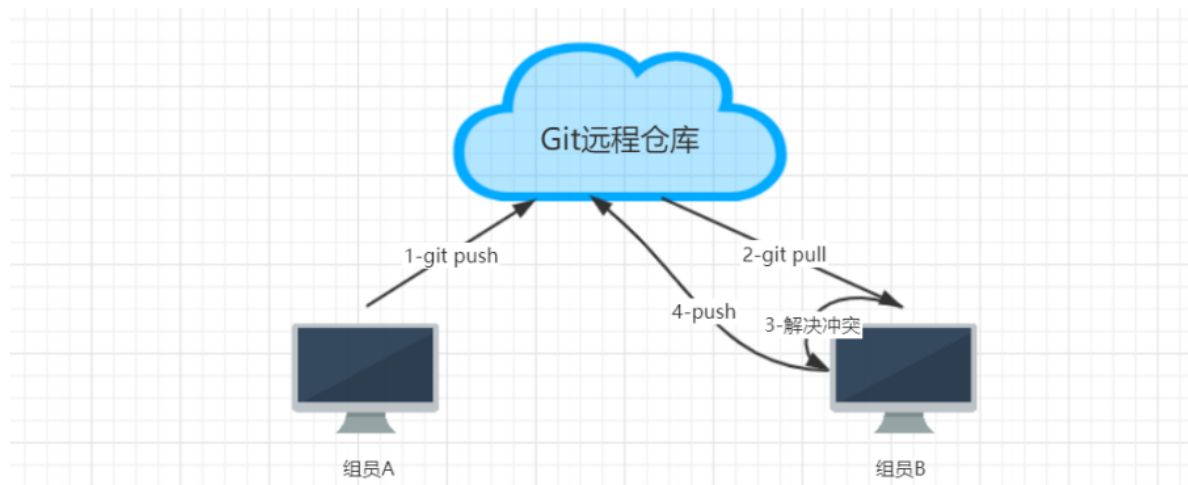
码。



4、组员A和组员B修改了同一个文件的同一行，提交到本地没有问题，但是推送到远程仓库时，后一个

推送操作就会失败。

解决方法：需要先获取远程仓库的代码到本地仓库，编辑冲突，提交并推送代码。

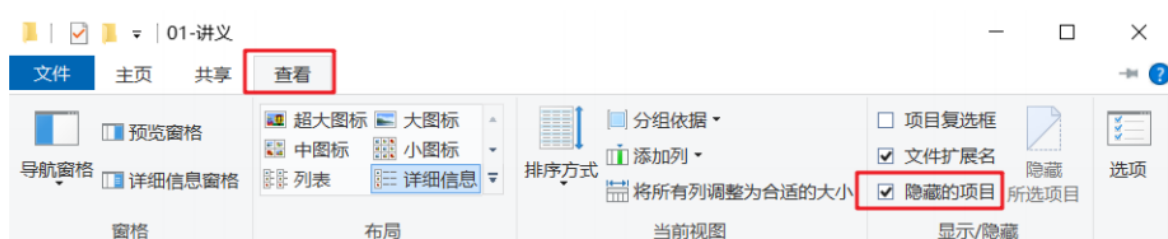


附:几条铁令

1. 切换分支前先提交本地的修改
2. 代码及时提交，提交过了就不会丢
3. 遇到任何问题都不要删除文件目录，第1时间找老师

附:疑难问题解决

1. windows下看不到隐藏的文件（.bashrc、.gitignore）



2. windows下无法创建.ignore|.bashrc文件

这里以创建 .ignore 文件为例：

- 在git目录下打开gitbash
- 执行指令 touch .gitignore

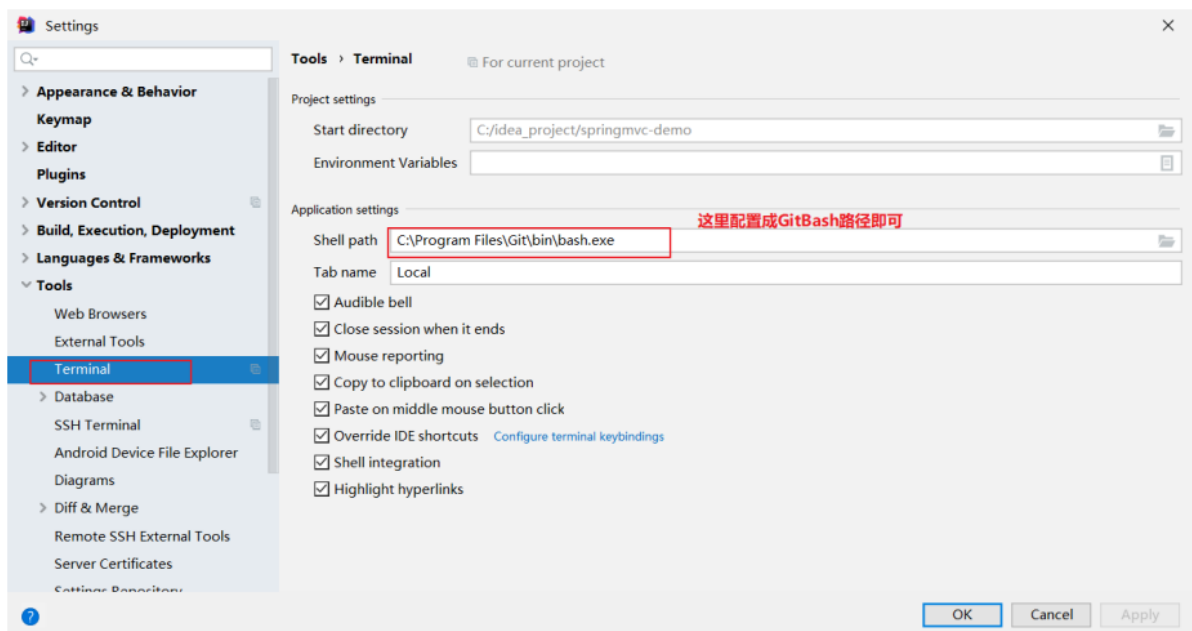
indows (C:) > idea_project > itheima-git-test >

名称	修改日期	类型	大小
.git	2020/7/14 17:13	文件夹	
.idea	2020/7/14 17:21	文件夹	
hello-world	2020/7/14 16:48	文件夹	
.gitignore	2020/7/14 17:23	文本文档	0 KB

MINGW64:/c/idea_project/itheima-git-test

```
zhang@zhangmeng MINGW64 /c/idea_project/itheima-git-test (dev2)
$ touch .gitignore
zhang@zhangmeng MINGW64 /c/idea_project/itheima-git-test (dev2)
$
```

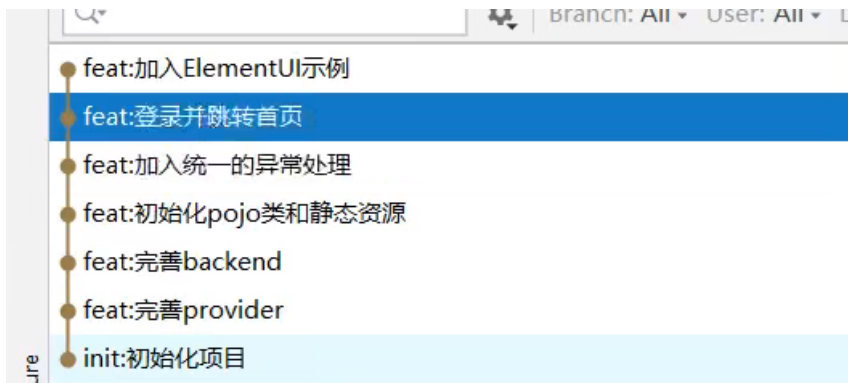
附：IDEA集成GitBash作为Terminal



Git的优势

name	代码获取	代码使用	存储空间
GIT	每次课只需要执行git pull即可	1-无需每次导入。 2-可以看到每个功能实现步骤的代码细节。 3-学生代码和老师代码在一个项目里，方便查看对照。	节省空间
拷贝	每次都要copy	1-每次使用都需要用idea来导入，使用极不方便。 2-学生拿到的代码是一个项目，包含了当堂课和前面课时的代码总和，不容易分清那部分代码是当天的代码。 3-老师的代码和学生代码在不同项目里，所有需要打开多个IDEA进行代码参考。	浪费空间

查看项目实现的步骤



分支模型

相当于老板或者组长给你发布任务

组长将原始项目（项目框架）发到Git共享版本库中，组员就去Git上将代码更新到本地

对于分支的使用，我们每天可以创建一个分支来存放今天工作的代码，然后完成工作后就把每天分支上的代码合并到master上，然后上传到共享版本库中，公司使用的是git-lab

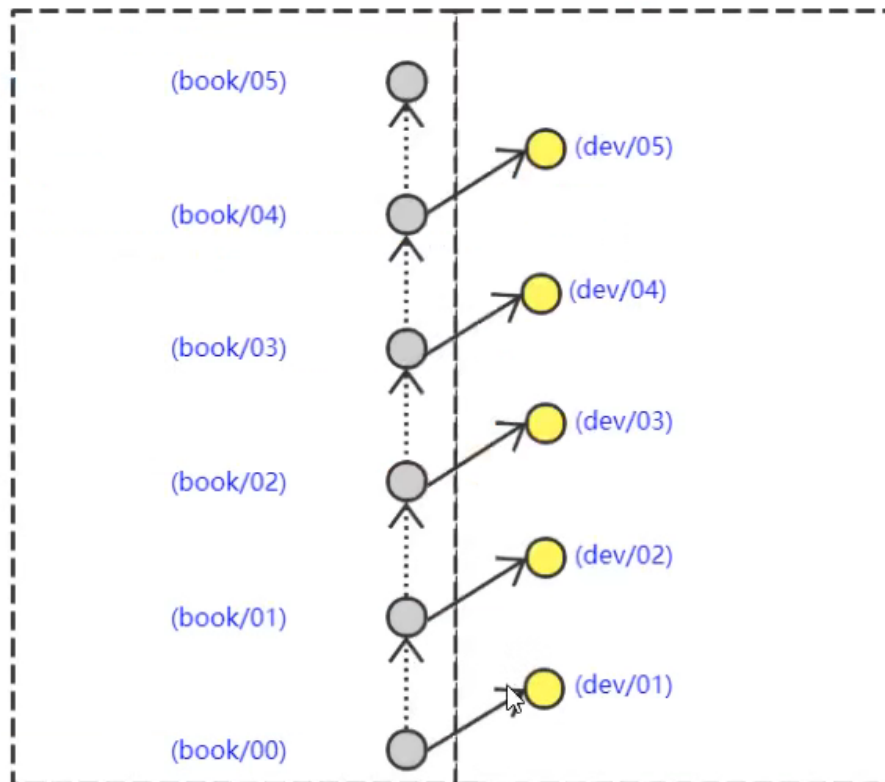
2. 分支模型

核心思想：

- 课前：
 - 老师：每次课前都会给学生共享1个分支（推送到远程仓库）
 - `book/xx` 分支
 - 这是本次课要实现的代码分支，和讲义的代码完全一致。
 - 里面的代码提交记录，详细描述了每一次对代码修改的细节。
 - 学生：将代码更新到本地（也就是 `git pull` 拉取到本地）
- 课上：（老师和学生操作完全一样，模仿老师即可）
 - 执行 **fetch** 操作（可以使用IDEA可视化操作，也可以使用命令操作）
 - 创建一个本次课的开发分支，
 - 格式： `dev/xx`
 - 从上一次课的实现代码上创建本次课的代码，例如：
 - 第1天，就从 `book/00` 创建一个开发分支 `dev/01`
 - `book/00` 是我们项目的初始代码分支
 - 第2天，就从 `book/01` 创建一个开发分支 `dev/02`
 -
 - 第10天，就从 `book/09` 创建一个开发分支 `dev/10`

讲义代码

课堂代码 (老师/学生)



每一个讲义代码都是事先准备好的，他包括了前面代码的总和

总结

今日总结：

- 工作区-》暂存区 `git add .`
- 暂存区-》仓库 `git commit -m 'commit message 01'`
- 查看状态 `git status`
- 查看提交记录 `git log` `git-log`
- 版本回退（理解即可） `git reset --hard <commitID>`
- 查看分支 `git branch`
- 创建并切换分支 `git checkout -b 分支名`
- 分支合并 `git merge 分支名`
 - 首先切换到目标分支上
- 远程仓库
 - `remote add`
 - `push`
 - `clone`
 - `fetch`
 - `pull (fetch+merge)`
- idea操作
 - 这是今天最重要的一个知识点。
 - 找到一个最适合你自己的操作方式。

idea操作Git有很多中方式，我们选择自己喜欢的一种就好了

